**Ministerul Educației și Cercetării al Republicii Moldova**
**IP Colegiul "Iulia Hasdeu" din Cahul**

# RAPORTUL
# STAGIULUI DE PRACTICĂ

Elevul: **Apareci Aurica**

Grupa: **AAW 2042**

Specialitatea: **Administrarea aplicațiilor web**

Baza de practică: **S.R.L. "BEND Technologies" mun. Chișinău**

Conducătorul stagiului de practică
de la unitatea economică

**Cazacu Cristian**

Conducătorul stagiului de practică
de la IP CIHC

**Tornea Victor**

Cahul **2024**

**CUPRINS**

**Introducere** ..... **3**

**Capitolul 1. Contextul profesional** ..... **5**

1.1 Caracteristica generală a companiei BEND Technologies ..... 5

1.2 Descrierea poziției de frontend developer ..... 5

**Capitolul 2. Descrierea proiectului** ..... **7**

2.1 O prezentare detaliată a proiectului Sumboard. ..... 7

2.2 Funcționalitățile principale și tehnologiile folosite. ..... 7

2.3 Instrumentele utilizate în procesul de dezvoltare ..... 11

**Capitolul 3. Metodologia Agile: Scrum** ..... **13**

3.1 Planificarea sprinturilor. Priorități. ..... 13

3.2 Ceremonii Scrum: Daily Stand-ups, Sprint Planning, Sprint Retrospective ..... 14

3.3 Beneficiile utilizării Scrum în proiectul Sumboard ..... 15

**Capitolul 4. Rolul și responsabilitățile** ..... **17**

4.1 Listarea sarcinilor și responsabilităților în timpul practicii. ..... 17

4.2 Experiența și realizările personale ..... 40

**Concluzii** ..... **42**

**Bibliografie** ..... **43**

# INTRODUCERE

*„Munca în echipă în dezvoltarea software este esențială.*
*Fiecare contribuție contează și aduce valoare proiectului final.”*

Caracteristica principală a activității de practică este integrarea efectivă a elevilor în activitatea unităților economice și instituțiilor de profil, în conformitate cu specialitatea lor de studiu. Acest proces de integrare le oferă elevilor oportunitatea de a aplica în mod practic cunoștințele teoretice acumulate în timpul cursurilor universitare și de a experimenta realitățile și provocările din mediul profesional. Practica desfășurată în cadrul companiei *Bend Technologies* a fost esențială pentru a aduce un plus de valoare formării mele profesionale, permițându-mi să contribui la proiecte reale și să colaborez cu profesioniști din domeniu.

Scopul principal al practicii ce anticipează probele de absolvire este atât aprofundarea și consolidarea cunoștințelor teoretice dobândite în timpul anilor de studii, cât și dezvoltarea competențelor profesionale obținute. În cadrul *Bend Technologies*, am avut ocazia să lucrez pe proiectul *Sumboard,* unde am aplicat în practică diverse tehnologii și metodologii învățate. Practica vizează integrarea elevilor în activitatea reală a unităților economice și instituțiilor de profil, facilitând astfel tranziția de la mediul academic la cel profesional.

Elevii își dezvoltă competențele profesionale concepute în cadrul practicilor de inițiere în specialitate și de specialitate, având ocazia să se familiarizeze cu procedeele, metodele și mijloacele noi de lucru utilizate în industrie. Sarcinile principale pe care trebuie să le îndeplinească elevul în perioada de practică includ aplicarea în practică a cunoștințelor obținute în cadrul cursurilor teoretice, aprofundarea și dezvoltarea competențelor la unitățile de curs de specialitate, acumularea experienței necesare pentru aplicarea procedeelor noi și elaborarea și întocmirea documentației de specialitate. Aceste activități au fost esențiale pentru pregătirea mea profesională și pentru realizarea cu succes a probei de absolvire.

*Structura raportului ce anticipează probele de absolvire* este determinată de obiectivele pe care aceasta le urmărește. Pentru a atinge scopul propus și pentru a răspunde la obiectivele enunțate, raportul va fi structurat în următoarele secțiuni:

*Introducerea* oferă o prezentare generală a contextului stagiului de practică. Practica de absolvire are ca obiectiv principal atât aprofundarea și consolidarea cunoștințelor teoretice dobândite în timpul anilor de studii, cât și dezvoltarea competențelor profesionale obținute. De asemenea, aceasta vizează integrarea elevilor în activitatea reală a unităților economice sau instituțiilor de profil, oferindu-le ocazia de a aplica în practică cunoștințele teoretice și de a acumula experiență practică relevantă.

În *Capitolul I*, se efectuează o analiză a companiei BEND Technologies, punându-se accent pe caracteristicile generale ale acesteia și pe descrierea poziției de frontend developer.

Această secțiune oferă o privire de ansamblu asupra companiei și a rolului meu în cadrul proiectului, evidențiind importanța și responsabilitățile specifice ale acestui rol.

*Capitolul II* oferă o prezentare detaliată a proiectului Sumboard, descriind funcționalitățile principale și tehnologiile folosite. În această secțiune se vor detalia obiectivele proiectului. De asemenea, se vor descrie instrumentele utilizate în procesul de dezvoltare, inclusiv platformele și mediile de dezvoltare, sistemele de versionare și colaborare și alte instrumente relevante.

*Capitolul III* este dedicat metodologiei Agile, cu un accent special pe Scrum. În această secțiune se va detalia planificarea sprinturilor, prioritățile și ceremoniile Scrum, cum ar fi Daily Stand-ups, Sprint Planning și Sprint Retrospective. De asemenea, se vor evidenția beneficiile utilizării Scrum în proiectul Sumboard, subliniind modul în care această metodologie a contribuit la eficiența și succesul proiectului.

*Capitolul IV* descrie rolul și responsabilitățile mele ca frontend developer în cadrul proiectului Sumboard. Această secțiune va lista sarcinile și responsabilitățile principale pe care le-am avut în timpul practicii, evidențiind activitățile specifice și contribuțiile mele. De asemenea, se va reflecta asupra experienței și realizărilor personale, subliniind dezvoltarea competențelor profesionale și lecțiile învățate.

În secțiunea *Concluzii* se prezintă concluziile generale ale stagiului de practică, subliniînd importanța practicii în formarea profesională și modul în care aceasta a contribuit la dezvoltarea mea ca frontend developer.

Prin structura și conținutul său, raportul urmărește să ofere o imagine clară și detaliată a experienței mele de practică, evidențiind atât aspectele teoretice cât și cele practice, și subliniind importanța acestei perioade în dezvoltarea mea profesională.

# Capitolul 1. CONTEXTUL PROFESIONAL

## 1.1 Caracteristica generală a companiei BEND Technologies

*Bend Technologies* este o platformă exclusivă care conectează dezvoltatori JavaScript pentru inițierea de afaceri inovatoare. Compania oferă echipe de dezvoltare complet dedicate, angajate în livrarea de produse software de la un capăt la altul, cu eficiență din punct de vedere al costurilor și fără a compromite calitatea. În plus, Bend se poziționează ca un partener de talent pentru clienții săi, oferind sprijin continuu de la începutul și până la finalul proiectului, asigurând o experiență fluidă și integrată.

## 1.2 Descrierea poziției de frontend developer

Poziția de frontend developer presupune responsabilitatea principală de a crea și implementa interfețele utilizator atractive și funcționale ale aplicațiilor web. Acesta colaborează strâns cu echipa de design pentru a transforma prototipurile și specificațiile în componente vizuale interactive, utilizând tehnologii precum HTML, CSS și JavaScript. De asemenea, frontend developerul asigură integrarea corectă a elementelor de frontend cu backend-ul aplicației, optimizând performanța și experiența utilizatorului.

Scopul principal al practicii desfășurate la Bend Technologies a fost să îmbunătățesc și să aplic în mod practic cunoștințele de dezvoltare front-end acumulate în timpul studiilor. *Proiectul Sumboard*, la care am lucrat, a necesitat implementarea și optimizarea componentelor de interfață utilizator, permițându-mi să îmi dezvolt abilitățile tehnice și să dobândesc experiență reală în cadrul unui proiect complex.

*Obiectivele specifice* ale practicii au inclus:

- Implementarea și optimizarea componentelor front-end utilizând tehnologii moderne conform cerințelor funcționale specificate de către project manager și team-lead.

- Colaborarea eficientă într-o echipă multidisciplinară, interacționând cu dezvoltatori backend și manageri de proiect pentru a asigura o integrare perfectă a funcționalităților.

- Contribuția la îmbunătățirea UI/UX a aplicației Sumboard, asigurând o experiență plăcută și intuitivă pentru utilizatori, în același timp eficientă din punct de vedere al arhitecurii.

- Adaptarea rapidă la noile tehnologii și metodologii utilizate în cadrul companiei, îmbunătățindu-mi astfel competențele și pregătindu-mă pentru provocările viitoare din domeniul dezvoltării web.

- Participarea la planificarea sprinturilor și atribuirea de story points pentru sarcinile din backlog. Aceasta a contribuit la înțelegerea procesului de dezvoltare Agile și la implicarea directă în organizarea și prioritizarea activităților din cadrul proiectului.

- Implicarea în procesul de code review pentru alți membri ai echipei, oferind feedback constructiv și asigurând calitatea codului. Acest obiectiv a contribuit la îmbunătățirea competențelor mele tehnice și la dezvoltarea abilităților de colaborare și comunicare în echipă.

- Participarea la evenimente de comunitate, cum ar fi Tech Coffee și Moldova Digital Summit, alături de CEO și echipa de marketing, pentru a prezenta publicului larg ideea proiectului Sumboard. Aceasta permite o înțelegere mai profundă a conceptului proiectului și facilitează interacțiunea directă cu clienții, ajutând la înțelegerea cerințelor acestora.

Perioada de practică desfășurată la Bend Technologies mi-a oferit oportunitatea de a pune în aplicare cunoștințele teoretice într-un mediu profesional, permițându-mi să îmi dezvolt competențele tehnice și să dobândesc experiență valoroasă în cadrul unui proiect real. Participarea la proiectul Sumboard m-a învățat importanța colaborării eficiente în echipă, a comunicării clare și a respectării standardelor de calitate. Am reușit să îmi îmbunătățesc abilitățile de front-end development prin utilizarea tehnologiilor moderne și prin adaptarea la noile metodologii și cerințe. Această experiență mi-a consolidat pasiunea pentru dezvoltarea web și mi-a oferit o bază solidă pentru viitoarele provocări profesionale în acest domeniu dinamic.

# Capitolul 2. DESCRIEREA PROIECTULUI

## 2.1 O prezentare detaliată a proiectului Sumboard.

În cadrul practicii, am avut oportunitatea de a activa în calitate de frontend developer la proiectul Sumboard. Proiectul Sumboard își propune să revoluționeze domeniul Embedded Analytics prin intermediul unui instrument low-code inovator, potrivit pentru soluții SaaS. Sumboard transformă datele din baza de date SaaS a clienților în rapoarte interactive, eliminând necesitatea cunoștințelor avansate de bază de date.

Sumboard oferă un proces de generare a analiticii care este atât robust, cât și flexibil, permițând utilizatorilor să conecteze multiple surse de date, să creeze tablouri de bord personalizate și să integreze aceste vizualizări direct în produsele lor, îmbunătățind astfel capacitatea de analiză și prezentare a datelor în cadrul organizațiilor. Cu ajutorul unui editor intuitiv de tablouri de bord *(figura 1.1)*, Sumboard oferă libertatea de a crea date analitice personalizate și interactive. În prezent, prioritatea echipei de proiect este dezvoltarea editorului, o componentă centrală a proiectului Sumboard. Acesta va oferi utilizatorilor posibilitatea de a personaliza tablouri de bord în cadrul organizației lor, precum și de a genera rapoarte în format PDF și alte funcționalități esențiale.

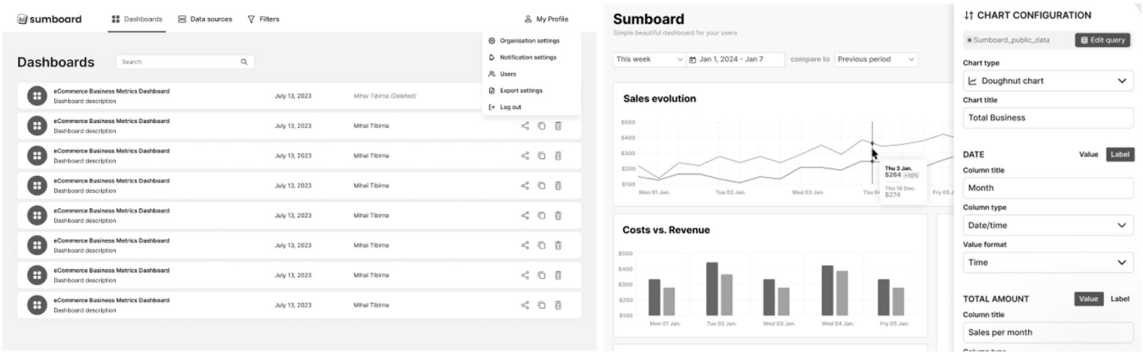



*Figura 1.1 Interfața editorului integrat în aplicația Sumboard*

Publicul țintă al proiectului Sumboard este format din companii care utilizează soluții SaaS și care doresc să extragă și să vizualizeze datele lor într-un mod simplu și eficient. Acesta include manageri de produs, analiști de date și alte persoane interesate de analize avansate, dar care nu au neapărat cunoștințe tehnice specializate.

## 2.2 Funcționalitățile principale și tehnologiile folosite.

Procesul de generare a analiticii în cadrul Sumboard se desfășoară în mai mulți pași bine definiți asigurând o experiență ușoară și eficientă pentru utilizatori. Primul pas constă în conectarea la diverse surse de date *(figura 2.1)*. Aceste surse pot fi baze de date SQL, API-uri sau documente, oferind astfel flexibilitate și posibilitatea de a accesa informații din multiple puncte de origine. Acest lucru permite utilizatorilor să centralizeze toate datele relevante într-un singur loc pentru o analiză cuprinzătoare.

După ce sursele de date sunt conectate, utilizatorii folosesc editorul intuitiv drag-and-drop pentru a construi tablouri de bord analitice moderne și atractive *(figura 2.2)*. Editorul permite personalizarea completă a acestor tablouri de bord, astfel încât utilizatorii să poată crea vizualizări interactive care răspund cel mai bine nevoilor organizației lor. Funcționalitățile de bază includ adăugarea de grafice, tabele și alte elemente vizuale, toate fiind aranjate într-un mod intuitiv și accesibil.

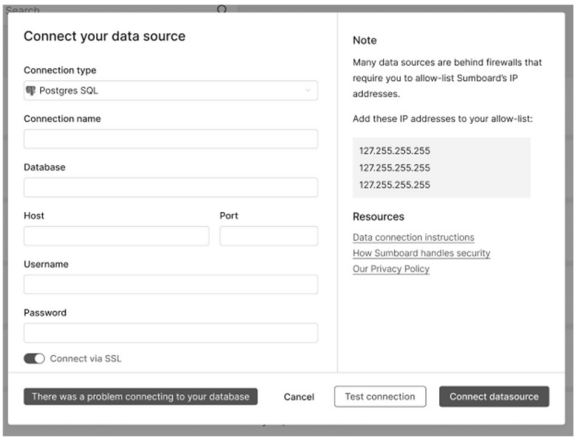



*Figura 2.1 Conectarea la sursa de date*
*docs.sumboard.io/getting-started/connect-your-data/*

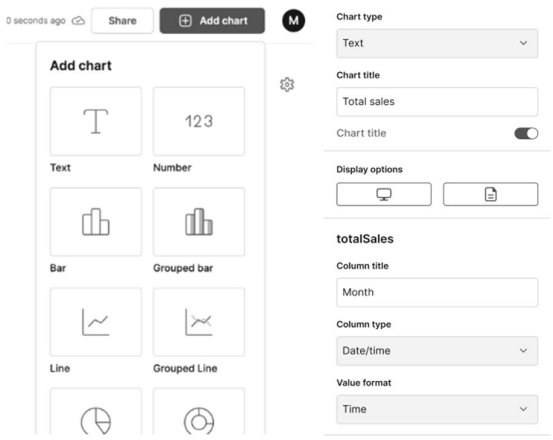
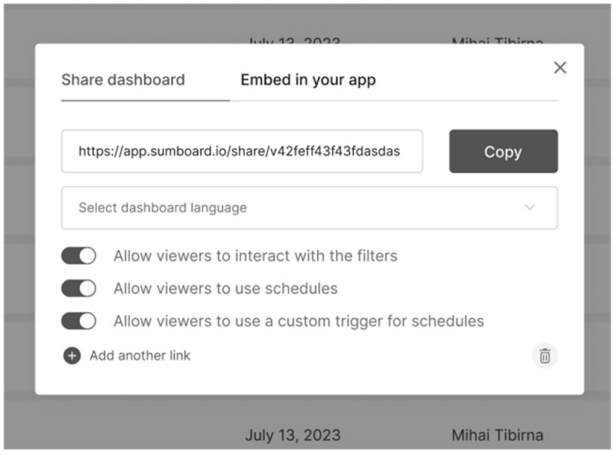



*Figura 2.2 Generarea dashboard-urilor*
*docs.sumboard.io/getting-started/work-with-data/*

Odată ce tablourile de bord sunt create, acestea pot fi integrate direct în produsul utilizatorilor, oferind o valoare adăugată semnificativă *(figura 2.3)*. Integrarea permite clienților să acceseze analitica direct din cadrul platformei lor, fără a fi nevoie să navigheze către alte aplicații sau servicii externe. Funcționalitățile incluse, cum ar fi graficele interactive, filtrele pentru date, opțiunile de comparație, exportul în format PDF/Excel și programarea rapoartelor, îmbunătățesc semnificativ experiența utilizatorului. Aceste caracteristici asigură nu doar o analiză detaliată și personalizată a datelor, dar și posibilitatea de a partaja și prezenta aceste date într-un format profesional și ușor de înțeles.



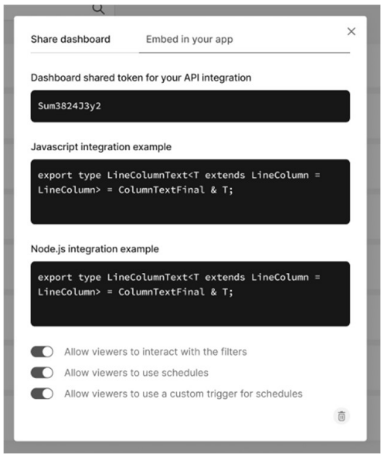




*Figura 2.3 Integrarea dashboard-urilor docs.sumboard.io/getting-started/share-embed/*

Panoul de administrare al Sumboard este un instrument esențial care oferă utilizatorilor controlul complet asupra configurărilor și setărilor la nivel de workspace. În cadrul secțiunii *Workspace Settings*, utilizatorii pot stabili setările implicite care se vor aplica tuturor dashboard-urilor create. *Secțiunea Users* permite gestionarea eficientă a utilizatorilor prin funcționalitatea de invitare a acestora în organizația Sumboard. Există două roluri distincte disponibile: Admin și Editor. Utilizatorul invitat va primi un email cu instrucțiuni pentru a se alătura organizației pe Sumboard.

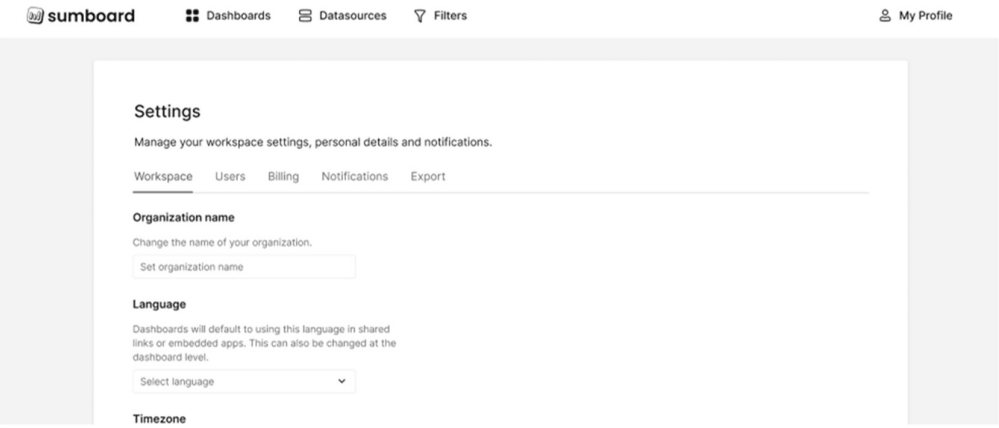



*Figura 2.4.1 Interfața Workspace Settings*

În *secțiunea Customization*, utilizatorii pot adapta aspectul dashboard-urilor pentru a se potrivi cu brandul sau stilul preferat. Sumboard oferă teme predefinite, dar și opțiunea de a crea teme personalizate. În secțiunea *Notification Settings*, utilizatorii pot gestiona și personaliza notificările prin email trimise prin Sumboard, permițând configurarea preferințelor de notificare pentru a îmbunătăți comunicarea. Sumboard permite, de asemenea, configurarea șabloanelor personalizate pentru emailuri, fișiere Excel și PDF pentru a personaliza aspectul rapoartelor. Aceste șabloane sunt încărcate la nivel de workspace. În *Profile Settings*, utilizatorii pot actualiza informațiile personale, cum ar fi numele și parola, pentru a menține securitatea și actualizarea contului.

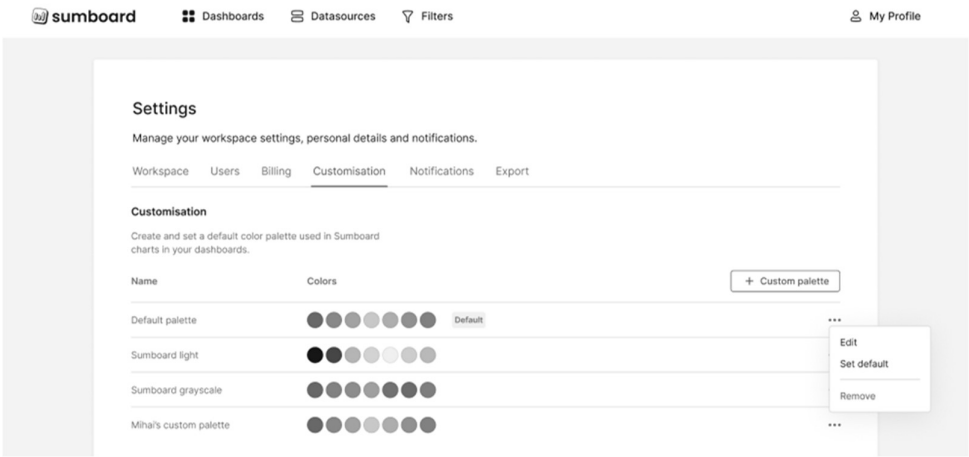



*Figura 2.4.2 Interfața Customisation Settings*

Pentru a asigura o experiență utilizatorului de cea mai înaltă calitate în cadrul Sumboard, sunt utilizate o gamă largă de tehnologii frontend moderne. *HTML* și *CSS* sunt fundamentale pentru structura și stilizarea paginilor web, oferind baza pentru design-ul atractiv și funcțional al editorului de tablouri de bord. Utilizarea *SCSS (Sassy CSS)* adaugă un nivel suplimentar de eficiență în gestionarea stilurilor, permițând utilizarea de variabile, mixins și nesting pentru a crea un cod CSS mai organizat și mai ușor de întreținut.

*Tailwind CSS* este integrat pentru a accelera procesul de dezvoltare și pentru a permite o personalizare rapidă și flexibilă a componentelor UI. Acest framework utilitar-first facilitează aplicarea stilurilor direct în markup, reducând astfel timpul necesar pentru stilizarea componentelor și asigurând un design coerent și modern.

*JavaScript* și *TypeScript* sunt utilizate pentru a adăuga interactivitate și funcționalități complexe aplicației Sumboard. JavaScript, fiind un limbaj versatil și bine cunoscut, permite dezvoltarea unor funcționalități dinamice, cum ar fi graficele interactive și filtrele de date. TypeScript, o extensie a JavaScript care adaugă tipuri statice, contribuie la scrierea unui cod mai robust și mai ușor de întreținut, prevenind erorile la timp de compilare și facilitând dezvoltarea în echipă.

*Angular*, un framework puternic pentru dezvoltarea de aplicații web, este utilizat pentru a construi arhitectura aplicației Sumboard. Angular permite dezvoltarea de componente reutilizabile și modulare, ceea ce este esențial pentru crearea editorului drag-and-drop și pentru integrarea funcționalităților complexe, cum ar fi exportul în PDF/Excel și programarea rapoartelor. Componentele în Angular sunt unități de bază care cuprind template-uri HTML, CSS și logică specifică, facilitând astfel gestionarea și reutilizarea codului. Views sunt structuri vizuale care se bazează pe aceste componente și care reprezintă diferite secțiuni ale interfeței aplicației, asigurând o navigare fluidă și o interacțiune optimă.

*Serviciile de translare* în Angular joacă un rol crucial în asigurarea unei aplicații multilingve, esențială pentru utilizatorii internaționali ai Sumboard. Aceste servicii permit traducerea dinamică a conținutului aplicației, astfel încât utilizatorii să poată interacționa cu aplicația în limba lor nativă. Utilizarea serviciilor de translare facilitează gestionarea și actualizarea traducerilor, asigurând totodată o experiență de utilizare consecventă și accesibilă pentru toate regiunile în care este utilizată aplicația.

Utilizarea acestor tehnologii frontend avansate și a arhitecturii modulare oferite de Angular permite dezvoltarea și menținerea funcționalităților complexe și interactive în cadrul Sumboard. Aceasta asigură un produs final robust, scalabil și ușor de utilizat, care satisface cerințele utilizatorilor și oferă o valoare adăugată semnificativă prin capabilitățile sale analitice și prin accesibilitatea globală.

## 2.3 Instrumentele utilizate în procesul de dezvoltare

În cadrul echipei de lucru, utilizarea unui set diversificat de instrumente moderne a fost esențială pentru asigurarea unei dezvoltări eficiente și colaborative. Aceste instrumente au permis echipei de dezvoltare să își desfășoare activitatea într-un mod structurat, organizat și integrat, maximizând productivitatea și calitatea produsului final.

Una dintre principalele platforme utilizate a fost *Visual Studio Code (VS Code),* un editor de cod sursă extrem de popular și flexibil. VS Code oferă suport pentru o multitudine de limbaje de programare și extensii, fiind ideal pentru dezvoltarea front-end. Integrarea facilă cu diverse plugin-uri, cum ar fi *ESLint* pentru linting și *Prettier* pentru formatarea codului, a permis menținerea unui cod curat și consistent. De asemenea, VS Code suportă debug-ul direct în editor, facilitând identificarea și rezolvarea rapidă a erorilor.

Pentru gestionarea versionării codului și colaborarea eficientă în echipă, am utilizat *Git* și *GitHub*. Git, un sistem de control al versiunilor distribuit, a permis echipei să lucreze simultan pe diverse funcționalități, având posibilitatea de a crea și gestiona ramuri (branches) pentru fiecare task sau feature în parte. GitHub a servit drept platformă centrală de depozitare a codului sursă și de colaborare, oferind facilități pentru pull requests, code reviews *(figura 2.5.1)*.

Pentru managementul task-urilor și al proiectelor, am folosit *Jira*. Acesta a fost esențial în planificarea și urmărirea progresului sprinturilor, oferind o vizualizare clară a sarcinilor individuale și a dependențelor dintre acestea. Jira a permis, de asemenea, o comunicare eficientă între membrii echipei și monitorizarea statusului fiecărui task, asigurând astfel un workflow agil și bine structurat *(figura 2.5.2)*.

*Slack* este o platformă de comunicare esențială pentru echipe, facilitând colaborarea prin canale tematice, mesaje directe și integrarea cu diverse instrumente de productivitate. În cadrul proiectului Sumboard, Slack a fost utilizat pentru coordonarea echipei, schimbul rapid de informații și rezolvarea promptă a problemelor.

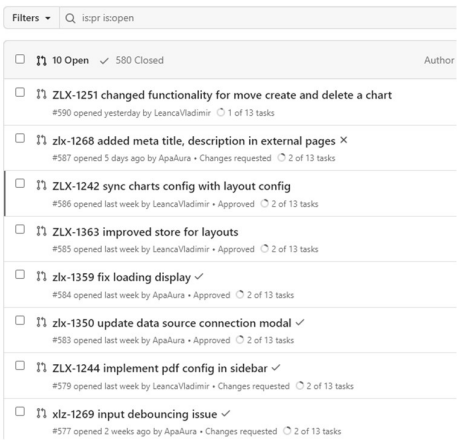



*Figura 2.5.1 Versionarea codului utilizând Git și GitHub*

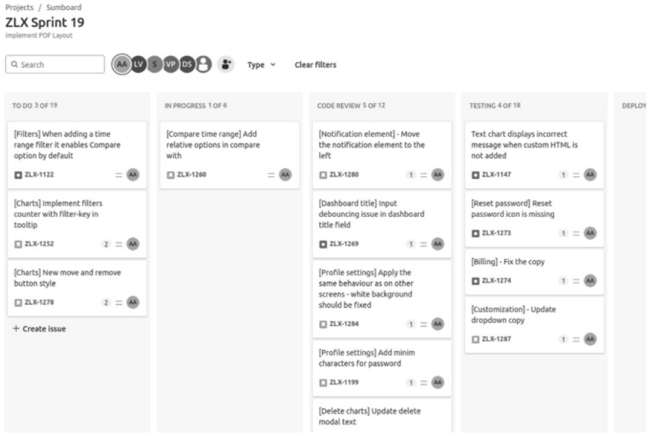



*Figura 2.5.2 Colaborarea în echipă utilizând platforma Jira*

*Figma* a fost un alt instrument crucial utilizat în dezvoltarea Sumboard, în special pentru designul UI/UX. Figma a permis designerilor și dezvoltatorilor să colaboreze în timp real, oferind un mediu comun pentru crearea și revizuirea prototipurilor de interfață. Această colaborare directă a accelerat procesul de design și a asigurat o implementare fidelă a componentelor vizuale. *(figura 2.5.3)*

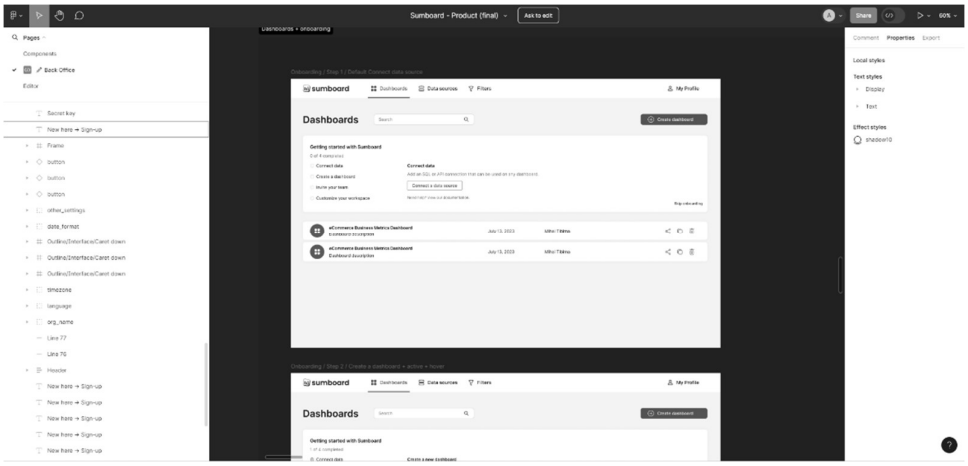



*Figura 2.5.3 Dezvoltarea designului UI/UX utilizând Figma*

*Swagger* a fost utilizat pentru documentarea și testarea API-urilor. Acesta a permis echipei să genereze automat documentația API-urilor și să efectueze teste directe din interfața Swagger, asigurând astfel că toate endpoint-urile sunt bine documentate și funcționează conform specificațiilor. *(figura 2.5.4)*




*Figura 2.5.4 Documentarea API-urilor utilizând Swagger*

# Capitolul 3. METODOLOGIA AGILE: SCRUM

## 3.1 Planificarea sprinturilor. Priorități.

*Scrum* este o metodologie Agile populară, utilizată pentru gestionarea și organizarea dezvoltării de proiecte complexe. Aceasta se bazează pe un cadru iterativ și incremental care permite echipelor să livreze software funcțional în mod continuu, adaptându-se rapid la schimbările cerințelor și priorităților. Scrum este structurat în sprinturi, care sunt cicluri de dezvoltare de obicei de 2-4 săptămâni, în cadrul cărora echipa își concentrează eforturile pe un set specific de funcționalități și sarcini. *(figura 3.1)*

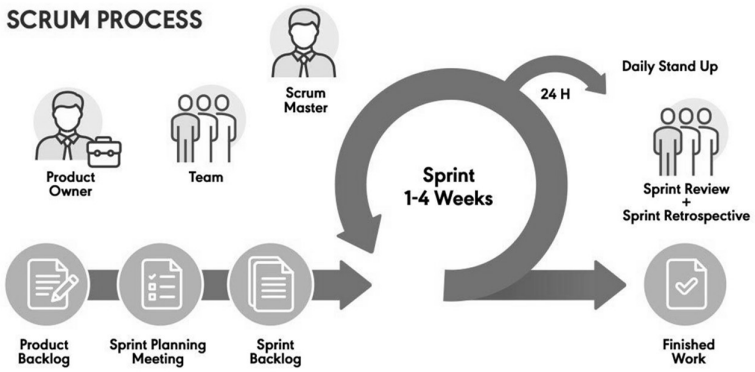



*Figura 3.1 Metodologia Agile: Scrum*

Planificarea sprinturilor este un pas esențial în procesul Scrum. În cadrul ședinței de planificare a sprintului, echipa Scrum stabilește obiectivele și prioritățile pentru următorul sprint, alegând elementele din backlog-ul produsului care vor fi abordate. Aceste elemente sunt detaliate în task-uri mai mici și atribuite membrilor echipei. Prioritizarea este esențială pentru a asigura că cele mai importante și valoroase funcționalități sunt dezvoltate mai întâi, asigurând astfel un impact maxim asupra produsului final.

Pe parcursul celor 8 săptămâni de practică ce anticipează probele de absolvire, am participat activ la patru sprinturi distincte, contribuind la diferite priorități ale proiectului:

| Activități / Sarcini de lucru | Termen planificat | Realizat |
|---|---|---|
| | Început/sfârșit | Numărul de zile |
| 1. ZLX Sprint 17 | 22.04 – 04.05 | 10 |
| Prioritate: Workspace Settings | | |
| 2. ZLX Sprint 18 | 14.05 – 24.05 | 10 |
| Prioritate: Customization | | |
| 3. ZLX Sprint 19 | 03.06 – 14.06 | 10 |
| Prioritate: Filters (Time Compare Range) | | |
| 4. ZLX Sprint 20 | 17.06 – 21.06 | 5 |
| Prioritate: Onboarding, Layouts | | |

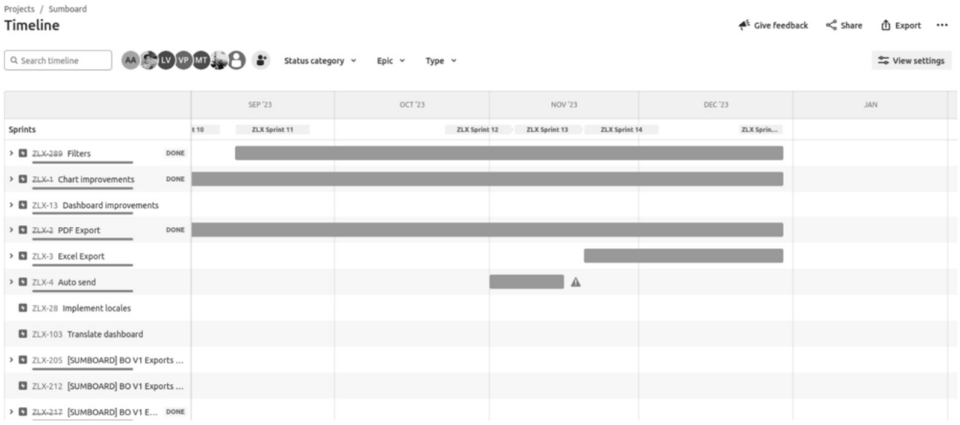



*Figura 3.2 Moniorizarea sprinturilor prin intermediul Jira*

## 3.2 Ceremonii Scrum: Daily Stand-ups, Sprint Planning, Sprint Retrospective

*Scrum* include mai multe ceremonii cheie care ajută echipa să rămână coordonată și să progreseze eficient. Aceste ceremonii oferă structura necesară pentru planificarea și evaluarea progresului, facilitând comunicarea și colaborarea între membrii echipei. Ele asigură, de asemenea, că toți participanții sunt aliniați la obiectivele comune și că obstacolele sunt identificate și rezolvate rapid.

*Daily Stand-ups:* Aceste întâlniri zilnice scurte (de obicei de 15 minute) permit membrilor echipei să își sincronizeze activitățile, discutând ce au realizat în ziua precedentă, ce vor face în ziua curentă și eventualele obstacole întâmpinate. Stand-up-urile ajută la identificarea rapidă a problemelor și la menținerea unui ritm constant de lucru.

*Sprint Planning:* În această ceremonie, echipa decide ce elemente din backlog vor fi abordate în următorul sprint. Se discută în detaliu fiecare element, se estimează efortul necesar și se stabilește un plan de acțiune pentru atingerea obiectivelor sprintului. Prioritățile sunt stabilite de către Product Owner, în funcție de valoarea adusă de fiecare element. *(figura 3.3)*

Atribuirea de *story points* în cadrul planificării sprinturilor este un proces esențial pentru evaluarea și gestionarea efortului necesar pentru realizarea diferitelor sarcini și funcționalități. În cadrul proiectului Sumboard, echipa utilizează story points pentru a estima complexitatea și timpul necesar pentru fiecare element din backlog. Aceste estimări nu se bazează doar pe numărul de ore de lucru, ci și pe factori precum incertitudinea, riscul și volumul de muncă. Prin atribuirea de story points, echipa poate prioritiza mai eficient sarcinile și poate asigura o distribuție echitabilă a muncii în cadrul sprintului. Acest proces facilitează, de asemenea, o planificare mai precisă și o prognoză mai bună a termenelor de livrare, contribuind astfel la succesul general al proiectului.

*Sprint Retrospective:* La sfârșitul fiecărui sprint, echipa se întâlnește pentru a discuta ce a mers bine, ce nu a mers bine și ce poate fi îmbunătățit în viitor. Această ceremonie oferă oportunitatea de a reflecta asupra performanței echipei și de a implementa îmbunătățiri continue

în procesul de dezvoltare. Prin această abordare, echipa poate ajusta metodele de lucru și poate aborda provocările într-un mod proactiv, asigurând astfel o eficiență sporită și o calitate superioară a produsului final.

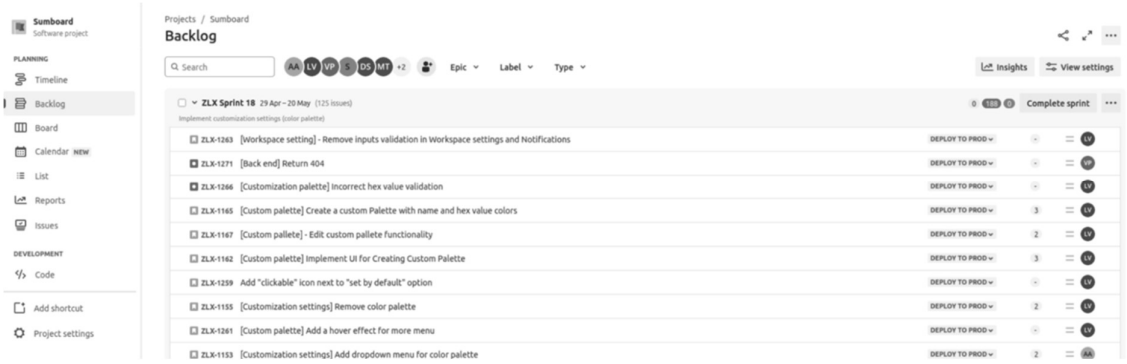



*Figura 3.3 Prioretizarea task-urilor din Backlog în cadrul Sprint Planning*

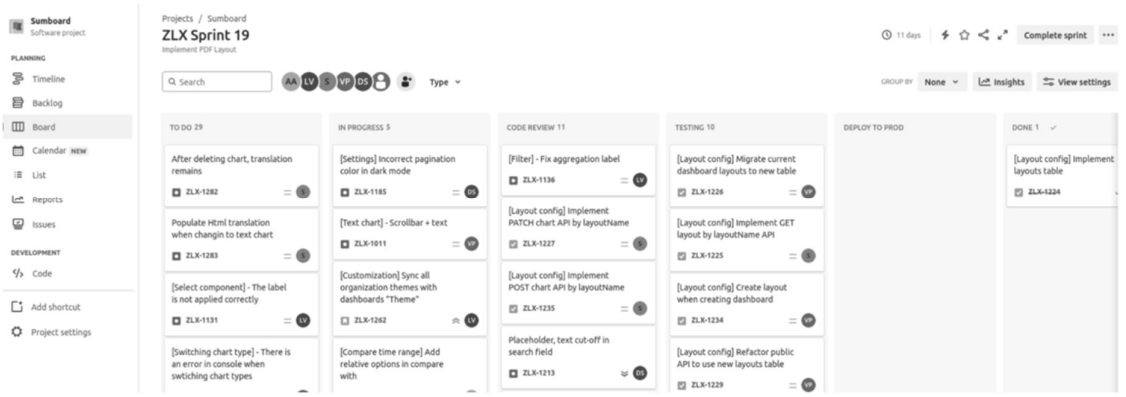



*Figura 3.4 Documentarea datelor din cadrul Sprint Planning utilizând Jira Software*

## 3.3 Beneficiile utilizării Scrum în proiectul Sumboard

Implementarea *metodologiei Scrum* în proiectul Sumboard a adus multiple beneficii, contribuind semnificativ la succesul dezvoltării produsului:

Scrum permite echipei să se adapteze rapid la schimbările cerințelor și priorităților, asigurând că produsul final este mereu aliniat cu nevoile utilizatorilor și ale pieței. Această flexibilitate este crucială în contextul unui startup, unde feedback-ul rapid și iterativ este esențial pentru succesul pe termen lung.

Ceremoniile Scrum, precum Daily Stand-ups și Sprint Reviews, împreună cu utilizarea unui backlog transparent, au oferit o *vizibilitate clară asupra progresului și a priorităților proiectului.* Acest lucru a facilitat o comunicare eficientă între membrii echipei și stakeholderi, asigurând că toată lumea este aliniată și informată cu privire la stadiul proiectului.

Sprint retrospectives au oferit echipei oportunitatea de a *învăța din fiecare sprint și de a implementa îmbunătățiri continue*. Acest proces a dus la o creștere constantă a eficienței și a calității produsului, deoarece echipa putea identifica rapid problemele și implementa soluții pentru a le remedia în următoarele cicluri de dezvoltare.

*Prioritizarea elementelor din backlog* pe baza valorii aduse a asigurat că echipa lucrează întotdeauna la cele mai importante și valoroase funcționalități pentru utilizatori și pentru succesul general al proiectului. Acest aspect a fost esențial pentru a maximiza resursele disponibile și pentru a livra funcționalități cheie într-un timp scurt, contribuind la atractivitatea și competitivitatea produsului pe piață.

În concluzie, utilizarea metodologiei Scrum în cadrul proiectului a facilitat o abordare structurată și eficientă a dezvoltării, aducând beneficii semnificative în ceea ce privește flexibilitatea, transparența, îmbunătățirea continuă și focalizarea pe valoare. Aceste beneficii au fost esențiale pentru succesul proiectului și pentru livrarea unui produs de înaltă calitate.

# Capitolul 4. ROLUL ȘI RESPONSABILITĂȚILE

## 4.1 Listarea sarcinilor și responsabilităților în timpul practicii.

Postul de frontend developer cuprinde o gamă largă de responsabilități care sunt esențiale pentru succesul oricărui proiect web. În cadrul practicii desfășurate la *Bend Technologies*, aceste responsabilități au inclus: contribuții la dezvoltarea UI/UX, optimizarea performanței, rezolvarea problemelor de frontend, colaborarea în echipă și îmbunătățirea continuă a cunoștințelor, participarea la planificarea sprinturilor, implicarea în procesul de code review. Acest capitol oferă o descriere detaliată a rolurilor și responsabilităților pe care le-am avut în cadrul practicii desfășurate la *Bend Technologies*. Aici sunt prezentate diversele sarcini la care am contribuit în timpul dezvoltării proiectului Sumboard, evidențiind activitățile specifice și competențele dobândite. Prin descrierea sarcinilor, se urmărește evidențierea modului în care am aplicat cunoștințele teoretice în practică și a impactului pe care l-am avut asupra proiectului pe parcursul practicii ce anticipează probele de absolvire.

Una din responsabilitățile de bază au constituit implementarea și optimizarea componentelor front-end utilizând tehnologii moderne conform cerințelor funcționale specificate de către project manager și team-lead. Am contribuit la ***dezvoltarea/actualizarea interfeței utilizatorului***, asigurându-mă că aceasta este atractivă și funcțională. Am utilizat tehnologii moderne precum HTML, CSS, SCSS, Tailwind, și framework-uri precum Angular pentru a crea o interfață intuitivă și eficientă.

**Exemplul nr. I ZLX-1345 [Sign in /Sign out] Update Sign-up, Sign-in UI**

În cadrul sarcinilor ce țin de dezvoltarea interfeței utilizatorului, procesul începe cu analiza detaliată a sarcinii în Jira, unde sunt definite criteriile de acceptare (acceptance criteria).

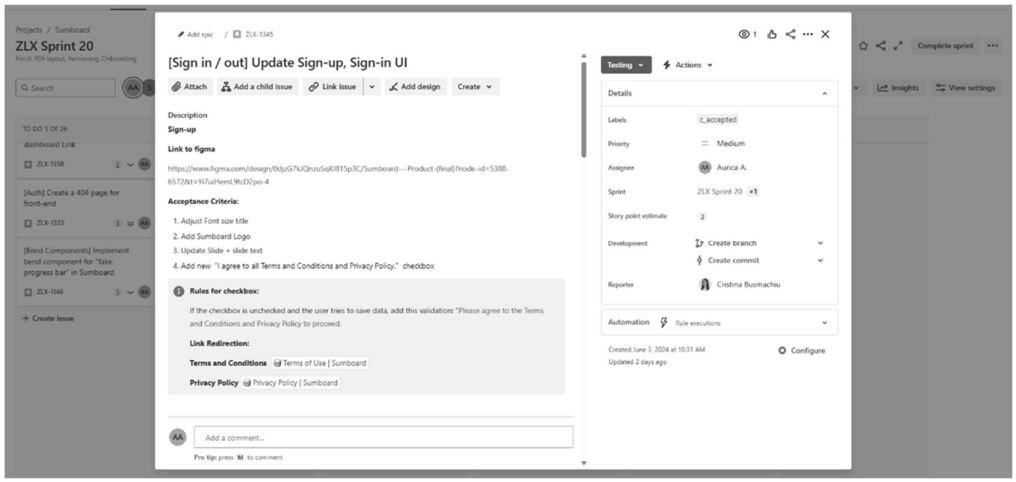



*Figura 4.1.1 Analiza ZLX-1345 [Sign in /Sign out] Update Sign-up, Sign-in UI în Jira Software*

Urmează vizualizarea designului în *Figma*, care oferă o reprezentare clară a elementelor vizuale ce trebuie implementate. După această etapă, se creează un branch nou în sistemul de versionare și se începe codarea propriu-zisă.

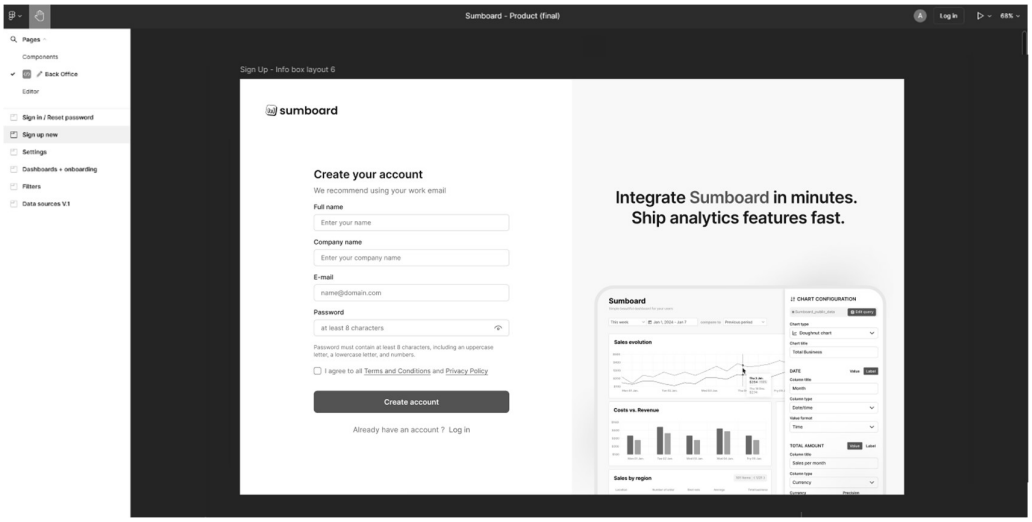



*Figura 4.1.2 Analiza designului în Figma*

De exemplu, pentru task-ul *ZLX-1345 "[Sign in / out] Update Sign-up, Sign-in UI"*, a fost necesară modificarea unui component din biblioteca *"bend-components"*. În acest context, am adăugat mesaj de eroare pentru componenta *"bend-checkbox",* am deschis un pull request, iar după ce acesta a fost integrat (merged), am creat un tag nou și l-am instalat în Sumboard.

projects/components/src/lib/checkbox/checkbox.component.html

```
@@ -10,4 +10,12 @@
      <ng-content></ng-content>
    </span>
  </label>
+
+ <div error kcErrors multiple #error="kcErrors" [@staggerChild]="error.stagger" [staggerTime]="140">
+   <ng-container *ngFor="let error of errors | keyvalue: sort">
+     <div @toggleY *kcError="error.key; label: error.value; let value; let error = error" class="error-field">
+       {{ value }}
+     </div>
+   </ng-container>
+ </div>
</bend-form-field-simple>
```

projects/demo/src/app/views/checkbox/checkbox.component.html

```
@@ -11,6 +11,26 @@

  <bend-checkbox start formControlName="empty" label="Start checkbox">This checkbox label start from top</bend-checkbox>

+ <!--errors-->
+ <h2>Errors</h2>
+ <bend-checkbox
+   label="Without control"
+   [checked]="false"
+   formControlName="require"
+   [errors]="{ required: 'This field is required' }"
+ >
+   Default checkbox
+ </bend-checkbox>
+
+ <bend-checkbox
+   formControlName="require"
+   label="Default checkbox"
+   formControlName="require"
+   [errors]="{ required: 'This field is required' }"
+ >
```

added errors to checkbox #128

Merged kryshac merged 1 commit into alpha from added-errors-to-checkbox 2 weeks ago

Conversation 1 Commits 1 Checks 7 Files changed 19 +105 −30

ApaAura commented 2 weeks ago

Compact checkbox

Compact checkbox

Start checkbox

This checkbox label start from top

Errors

Without control

Default checkbox

This field is required

Reviewers: kryshac

Assignees: No one—assign yourself

Labels: None yet

Projects: None yet

În Git, *tagurile* sunt utilizate pentru a marca anumite puncte din istoria repository-ului cu semnificație specială. Acestea sunt cel mai des utilizate pentru a marca versiuni (release-uri) specifice ale codului sursă. Mai jos este ilustrată crearea tagului *"0.0.1-alpha.105"*, care ulterior mi-a permis să integrz ultima versiune a bibliotecii *"bend-components"* în proiect.

terminal

```
user@user-Latitude-5540:~/Desktop/ng-components$ git tag -a "0.0.1-alpha.105"
user@user-Latitude-5540:~/Desktop/ng-components$ git push origin 0.0.1-alpha.105
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 178 bytes | 178.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:BEND-Technologies/ng-components.git
 * [new tag]         0.0.1-alpha.105 -> 0.0.1-alpha.105
```

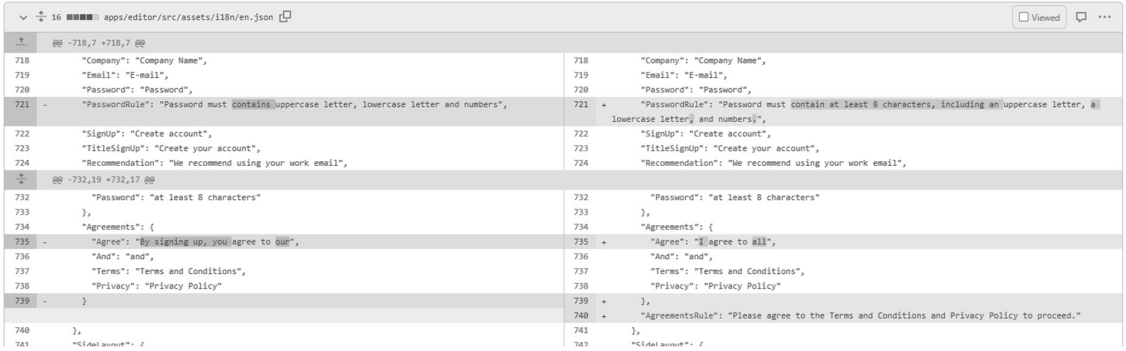



Conform cerințelor specificate în task-ul *ZLX-1345 "[Sign in / out] Update Sign-up, Sign-in UI",* am modificat interfața conform designului specificat. Am creat componentele necesare folosind comanda *ng g c <nume>,* am stilizat aceste componente și am adăugat translări pentru tot conținutul. Imaginea a fost adăugată cu *ngSRC*, ceea ce a permis o încărcare mai eficientă și o mai bună gestionare a resurselor media.









După finisarea implementării, am realizat *git squash* pentru a combina commit-urile într-unul singur, asigurând astfel un istoric de commit-uri mai curat. Ulterior, am publicat branch-ul pe remote și am deschis un pull request, completând totodată o anchetă pentru a facilita un code review mai rapid *(figura 4.1.3)*. După ce am primit comentarii din partea echipei, am fixat toate observațiile și pull request-ul a fost merge-uit de către team lead.

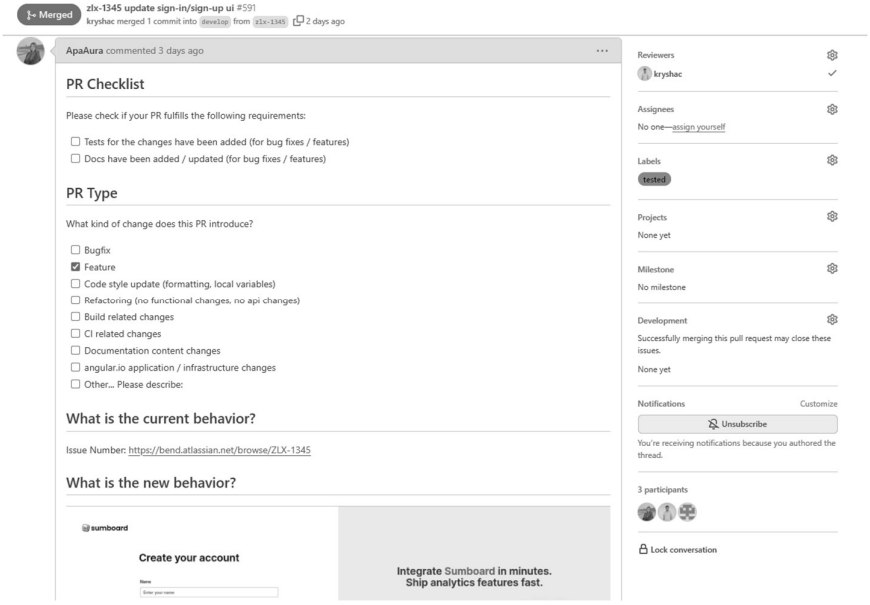



*Figura 4.1.3 PR Checklist pentru facilitarea code review în cadrul echipei*

**Exemplul nr. II ZLX-1069 [Profile Settings] Implement the profile UI**
**ZLX-1073 [Profile Settings] Implement business logic**

Un alt exemplu semnificativ al activității mele în cadrul practicii la *Bend Technologies* a fost implementarea task-urilor *ZLX-1069 "[Profile Settings] Implement the profile UI"* și *ZLX-1073 "[Profile Settings] Implement business logic"*. Aceste sarcini au fost incluse în prioritățile sprintului *ZLX-17* și au implicat crearea paginii de profil unde utilizatorii își pot actualiza datele personale, precum numele și parola. Am dezvoltat interfața utilizator (UI) pentru setările de profil, asigurându-mă că designul este intuitiv și accesibil. În plus, am implementat logica de business necesară pentru a gestiona actualizarea datelor de profil, integrând corect aceste funcționalități cu backend-ul pentru a asigura securitatea și integritatea informațiilor utilizatorilor.




*Figura 4.2.1 Analiza ZLX-1069 și ZLX-1073 în Jira Software*

Am vizualizat designului în *Figma*, care oferă o reprezentare clară a elementelor vizuale ce trebuie implementate. *(figura 4.2.2)* Am creat un branch nou în sistemul de versionare și am creat componentele necesare implementării interfeței.

Pentru a realiza sarcinile, am analizat *datele API* pentru toate metodele relevante utilizând *Swagger*.*(figura 4.2.3)* Swagger este un instrument puternic utilizat pentru a proiecta, construi, documenta și utiliza API-uri RESTful. Acesta permite dezvoltatorilor să vizualizeze și să interacționeze cu API-urile lor fără a fi nevoie să scrie manual cod pentru solicitări și răspunsuri. Prin utilizarea Swagger, m-am asigurat că înțeleg pe deplin interacțiunile și cerințele backend-ului, ceea ce mi-a permis să integrez eficient funcționalitățile necesare în interfață.

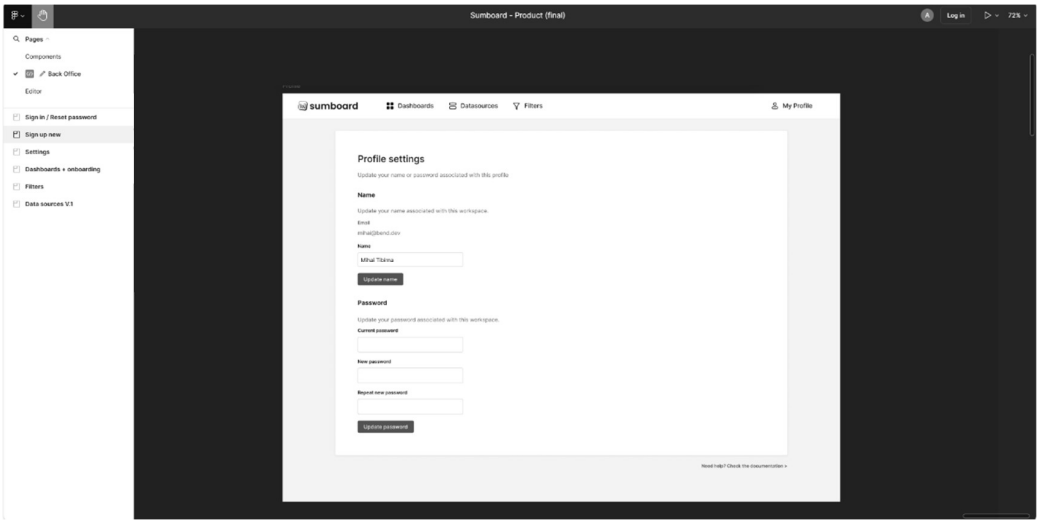



*Figura 4.2.2 Analiza designului în Figma*




*Figura 4.2.3 Analiza API-urilor utilizând Swagger*

Acest exemplu evidențiază abilitatea mea de a lucra eficient atât pe partea de UI, cât și pe partea de logică de business, contribuind astfel la îmbunătățirea experienței utilizatorului în cadrul proiectului.

23 ■■■■■■ apps/editor/src/app/views/profile/components/password/password.component.ts

Viewed

@@ -0,0 +1,23 @@

```
+ import { ChangeDetectionStrategy, Component, Input } from '@angular/core';
+
+ import { WrappedFormControl, provideValueAccessor } from '@k5cjs/forms';
+
+ @Component({
+   selector: 'app-password',
+   templateUrl: './password.component.html',
+   styleUrls: ['./password.component.scss'],
+   changeDetection: ChangeDetectionStrategy.OnPush,
+   providers: [provideValueAccessor(PasswordComponent)],
+ })
+ export class PasswordComponent extends WrappedFormControl {
+   @Input() label!: string;
+   @Input() error = '';
+
+   showPassword = false;
+   type: 'text' | 'password' = 'password';
+
+   togglePassword() {
+     this.showPassword = !this.showPassword;
+     this.type = this.showPassword ? 'text' : 'password';
+   }
+ }
```

11 ■■■■■■ apps/editor/src/app/views/profile/components/update-name/update-name.component.html

Viewed

@@ -0,0 +1,11 @@

```
+ <h4>{{ 'Name' | translate }}</h4>
+ <div class="description">{{ 'NameDescription' | translate }}</div>
+
+ <div class="email">
+   <div class="email__label">{{ 'Email' | translate }}</div>
+   <div class="email__value">{{ user.email }}</div>
+ </div>
+
+ <bend-input [formControl]="name" class="input" [label]="'Name' | translate"></bend-input>
+
+ <bend-button (click)="update()" class="button">{{ 'UpdateName' | translate }}</bend-button>
```

30 ■■■■■■ apps/editor/src/app/views/profile/components/update-name/update-name.component.ts

Viewed

@@ -0,0 +1,30 @@

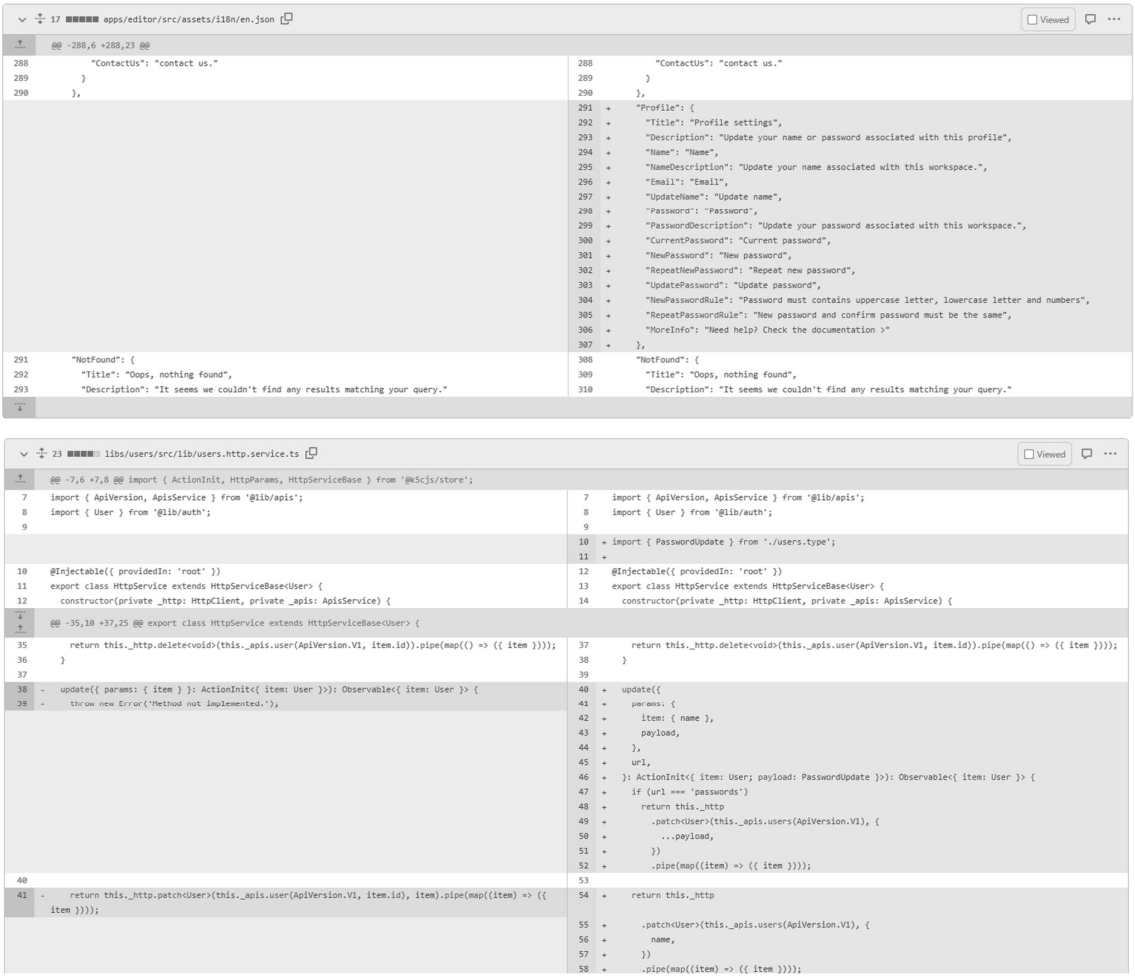
```
+ import { ChangeDetectionStrategy, Component, Input, OnInit } from '@angular/core';
+ import { NonNullableFormBuilder, Validators } from '@angular/forms';
+
+ import { User } from '@lib/auth';
+ import { FormOf } from '@lib/helpers';
+ import { UsersService } from '@lib/users';
+
+ @Component({
+   selector: 'app-update-name',
+   templateUrl: './update-name.component.html',
+   styleUrls: ['./update-name.component.scss'],
+   changeDetection: ChangeDetectionStrategy.OnPush,
+ })
+ export class UpdateNameComponent implements OnInit {
+   @Input() user!: User;
+
+   name!: FormOf<User['name']>;
+
+   constructor(private _users: UsersService, private _fb: NonNullableFormBuilder) {}
+
+   ngOnInit(): void {
+     this.name = this._fb.control(this.user.name, Validators.required.bind(this));
+   }
+
+   update(): void {
+     if (this.name.invalid) return;
+
+     this._users.update({ params: { item: { name: this.name.value, id: this.user.id } } });
+   }
+ }
```

54 ■■■■■■ apps/editor/src/app/views/profile/components/update-password/update-password.component.ts

Viewed

@@ -0,0 +1,54 @@

```
+ import { ChangeDetectionStrategy, Component, Input, OnInit } from '@angular/core';
+ import {
+   AbstractControl,
+   FormControl,
+   NonNullableFormBuilder,
+   ValidationErrors,
+   ValidatorFn,
+   Validators,
+ } from '@angular/forms';
+
+ import { PASSWORD_REGEX, User } from '@lib/auth';
+ import { FormOf } from '@lib/helpers';
+ import { PasswordUpdate, UsersService } from '@lib/users';
+
+ @Component({
+   selector: 'app-update-password',
+   templateUrl: './update-password.component.html',
+   styleUrls: ['./update-password.component.scss'],
+   changeDetection: ChangeDetectionStrategy.OnPush,
+ })
+ export class UpdatePasswordComponent implements OnInit {
+   @Input() user!: User;
+
+   form!: FormOf<PasswordUpdate>;
+
+   repeatPassword!: FormControl<string>;
+
+   constructor(private _users: UsersService, private _fb: NonNullableFormBuilder) {}
+
+   ngOnInit(): void {
+     this.form = this._fb.group({
+       currentPassword: this._fb.control('', Validators.required.bind(this)),
+       newPassword: this._fb.control('', [Validators.required.bind(this),
   Validators.pattern(PASSWORD_REGEX)]),
+     });
+
+     this.repeatPassword = this._fb.control('', [Validators.required.bind(this),
   this.samePassword().bind(this)]);
```




După ce am terminat implementarea, am efectuat *„git squash”* pentru a uni toate commit-urile într-un singur commit coerent, pentru a menține o istorie clară a modificărilor. Am apoi publicat branch-ul modificat pe repository-ul remote și am inițiat un pull request. Pentru a facilita un review rapid al codului, am completat o anchetă detaliată *(figura 4.2.4)*. După ce am primit feedback de la echipă, am abordat toate sugestiile și pull request-ul a fost integrat în branch-ul principal de către team lead.

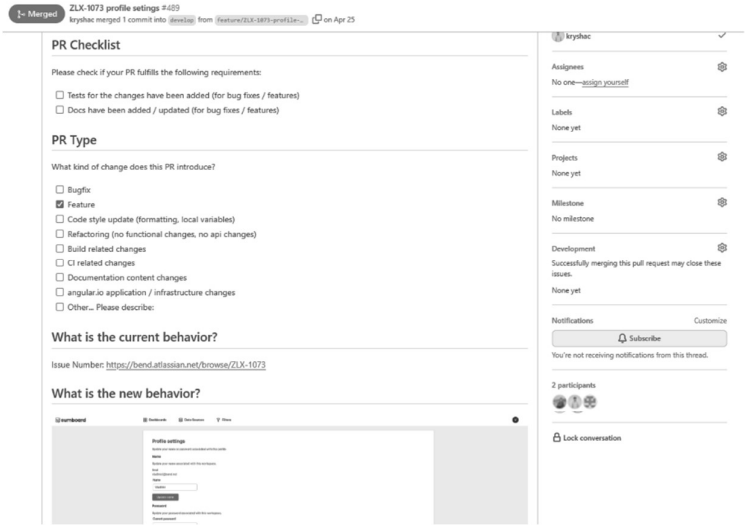



*Figura 4.2.4 PR Checklist pentru facilitarea code review în cadrul echipei*

**Exemplul nr. III**
**ZLX-1152 [Customization Settings] Implement UI for customization**
**ZLX-1153 [Customization Settings] Add dropdown menu for color palette**

Un alt exemplu de dezvoltare a interfeței utilizatorului a fost implementarea task-urilor *ZLX-1152 "[Customization Settings] Implement UI for customization"* și *ZLX-1153 "[Customization Settings] Add dropdown menu for color palette"* care au făcut parte din prioritățile sprintului *ZLX-18*. Aceste task-uri au implicat crearea unei pagini unde utilizatorii pot modifica paletele de culori aplicate la editarea chart-urilor și dashboard-urilor, adăuga palete personalizate, seta paleta implicită și șterge paletele existente. Am analizat designul în *Figma* pentru a înțelege cum trebuie să arate interfața și am creat componentele necesare pentru a respecta aceste specificații. *(figura 4.3.2)* De asemenea, am utilizat *Swagger* pentru a analiza datele API relevante, asigurându-mă că înțeleg pe deplin interacțiunile și cerințele backend-ului, astfel încât toate funcționalitățile să fie corect implementate și integrate.

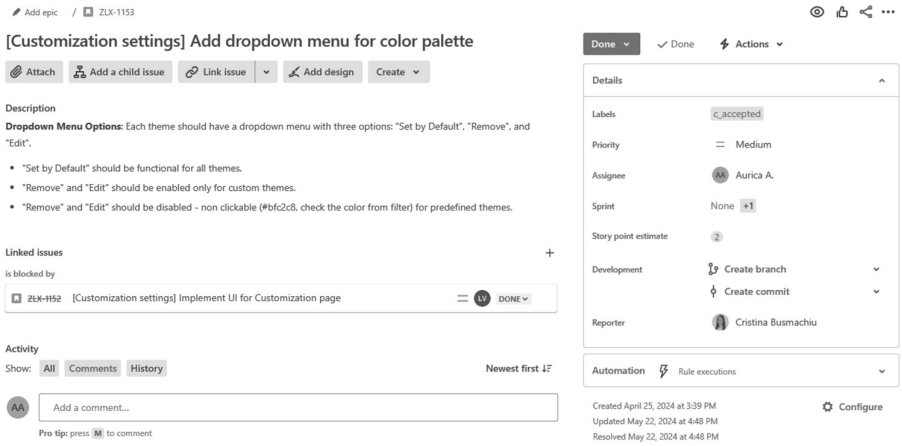



*Figura 4.3.1 Analiza ZLX-1152 și ZLX-1153 în Jira Software*

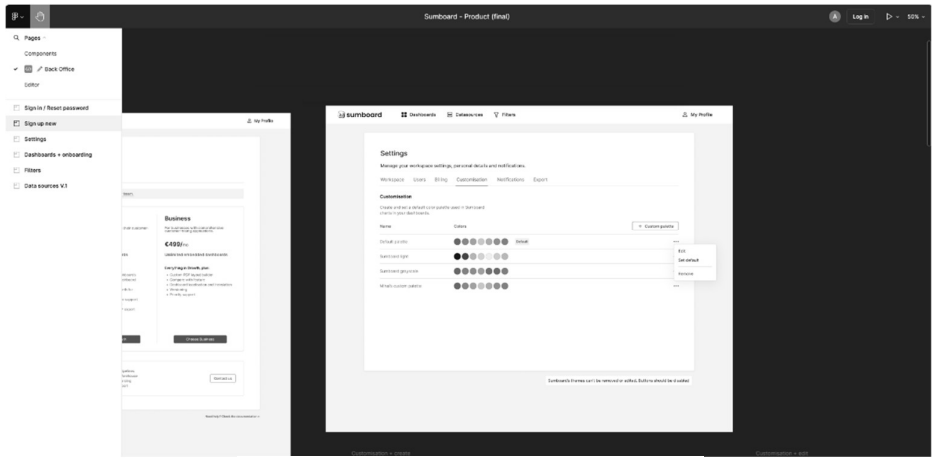



*Figura 4.3.2 Analiza designului în Figma*

După analiza sarcinilor și înțelegerea cerințelor din Jira, procesul de lucru începe cu actualizarea branch-ului principal, folosind comanda *git fetch* pe branch-ul *develop*. Urmează crearea unui branch nou pentru a lucra pe task-ul specific. Pentru a pregăti mediul de dezvoltare, rulez comanda *npm i* pentru a instala toate dependențele necesare *și npm run build:libs* pentru

a construi librăriile utilizate în proiect. Apoi, rulăm *npm run serve* pentru a porni serverul de dezvoltare și a verifica schimbările în timp real. După această pregătire, încep codarea efectivă, implementând funcționalitățile și componentele necesare conform specificațiilor. După finalizarea codării, urmează testarea riguroasă pentru a asigura că toate modificările funcționează corect și nu afectează alte părți ale aplicației. Acest proces metodic și structurat asigură o dezvoltare eficientă și de calitate.



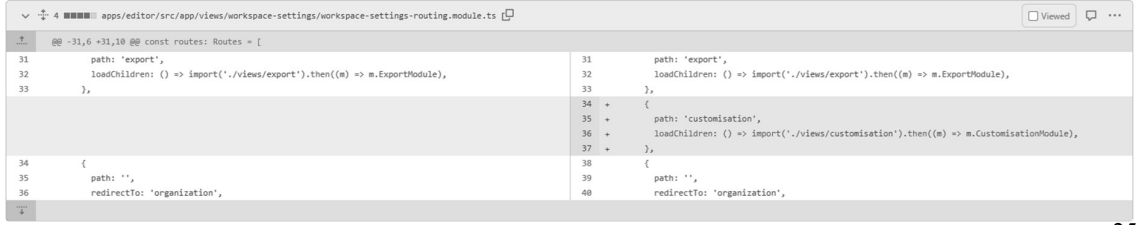
```
22 ■■■■■ ...kspace-settings/views/customisation/components/palettes-list/palettes-list.component.html
@@ -0,0 +1,22 @@
+ <div class="header">
+   <div class="header__item">{{ 'Name' | translate }}</div>
+   <div class="header__item">{{ 'Colors' | translate }}</div>
+   <div class="header__item">
+     <bend-button class="button" stroked>
+       <div class="button__container">
+         <kc-icon src="plus" class="icon"></kc-icon>
+         {{ 'CustomPalette' | translate }}
+       </div>
+     </bend-button>
+   </div>
+ </div>
+
+ <ng-container *ngIf="organization$ | async as organization">
+   <app-palette-default *ngFor="let palette of palettes" [palette]="palette"
  [organization]="organization" />
+
+   <ng-container *ngIf="palettesOrg$ | async as palettesOrg">
+     <ng-container *ngIf="palettesOrg.length">
+       <app-palette *ngFor="let palette of palettesOrg" [palette]="palette" [organization]="organization"
  />
+     </ng-container>
+   </ng-container>
+ </ng-container>
```



```
64 ■■■■■ ...orkspace-settings/views/customisation/components/palettes-list/palettes-list.component.ts
@@ -0,0 +1,64 @@
+ import { ChangeDetectionStrategy, Component } from '@angular/core';
+ import { Observable, map } from 'rxjs';
+
+ import { Organization, OrganizationService } from '@lib/organization';
+ import { Palette, PalettesService } from '@lib/palettes';
+ import { PaletteName, getPaletteByPaletteName } from '@lib/theme';
+
+ @Component({
+   selector: 'app-palettes-list',
+   templateUrl: './palettes-list.component.html',
+   styleUrls: ['./palettes-list.component.scss'],
+   changeDetection: ChangeDetectionStrategy.OnPush,
+ })
+ export class PalettesListComponent {
+   palettes!: Palette[];
+
+   palettesOrg$!: Observable<Palette[]>;
+
+   organization$!: Observable<Organization>;
+
+   constructor(private _palettes: PalettesService, private _organization: OrganizationService) {
+     this.organization$ = this._organization.getById({ params: { item: { id: '' } } }).pipe(map(({ item
  }) => item));
+
+     this.palettes = [
+       {
+         name: PaletteName.Default,
+         id: PaletteName.Default,
+         colors: {
+           light: getPaletteByPaletteName(PaletteName.Default).light,
```



```
22 ■■■■■ .../editor/src/app/views/workspace-settings/views/customisation/customisation.component.scss
Viewed
@@ -0,0 +1,22 @@
+ :host {
+   @apply
+     mt-6
+     text-on-secondary
+     flex
+     flex-col;
+ }
+
+ h3 {
+   @apply
+     font-semibold
+     text-base;
+ }
+
+ .description {
+   @apply
+     max-w-sm
+     mt-3
+     font-normal
+     text-sm
+     text-on-surface-variant;
+ }
```



```
4 ■■■■□ apps/editor/src/app/views/workspace-settings/workspace-settings-routing.module.ts
Viewed
@@ -31,6 +31,10 @@ const routes: Routes = [
        path: 'export',
        loadChildren: () => import('./views/export').then((m) => m.ExportModule),
      },
+     {
+       path: 'customisation',
+       loadChildren: () => import('./views/customisation').then((m) => m.CustomisationModule),
+     },
      {
        path: '',
        redirectTo: 'organization',
```

După finisarea implementării, am realizat *git squash* pentru a combina commit-urile într-unul singur, asigurând astfel un istoric de commit-uri mai curat. Ulterior, am publicat branch-ul pe remote și am deschis un pull request, completând totodată o anchetă pentru a facilita un code review mai rapid *(figura 4.3.3)*. După ce am primit comentarii din partea echipei, am fixat toate observațiile și pull request-ul a fost merge-uit de către team lead.

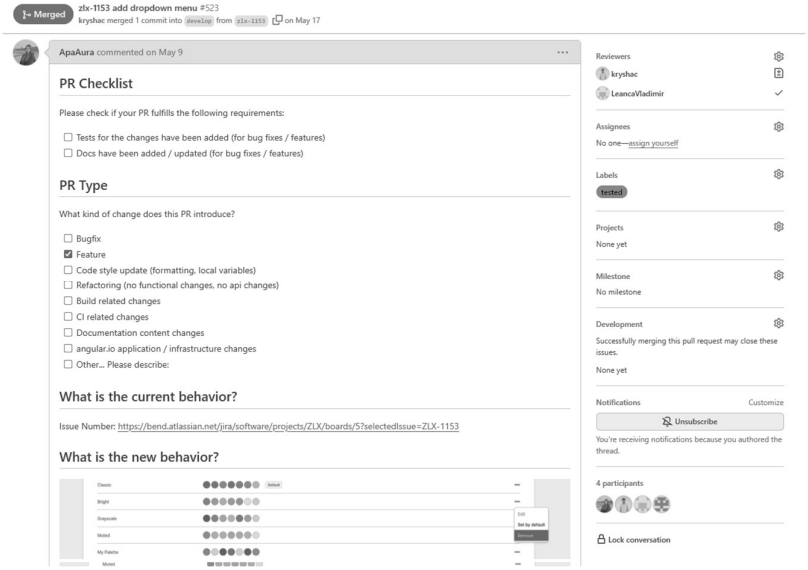



*Figura 4.3.3 PR Checklist pentru facilitarea code review în cadrul echipei*

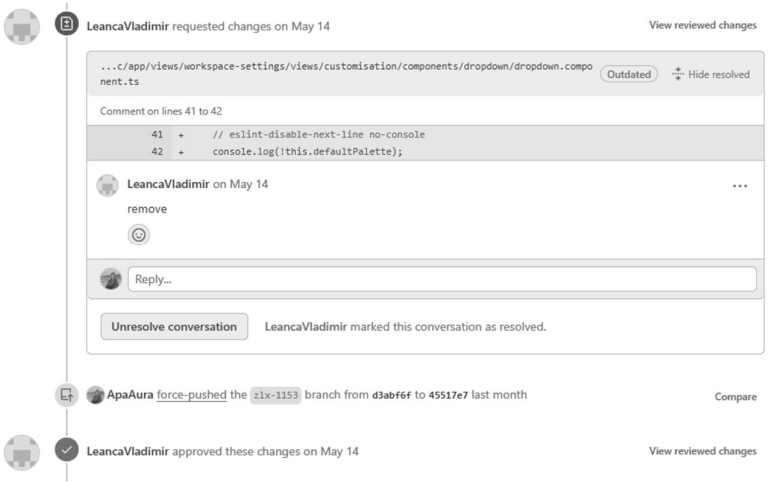



*Figura 4.3.4 Fixarea comentariilor în urma code review*

Pe parcursul practicii ce anticipează probele de absolvire, una din responsabilități a fost ***implementarea librăriilor, adăugând funcționalități noi și optimizând performanța aplicației.*** De asemenea, am integrat componentele front-end cu API-urile backend, asigurându-mă că datele sunt transmise și recepționate corect, configurând și testând punctele de acces API pentru a asigura o comunicare fluentă între front-end și back-end. În plus, am identificat și implementat îmbunătățiri pentru optimizarea performanței aplicației Sumboard, contribuind astfel la îmbunătățirea vitezei și a reactivității generale a aplicației. Aceste responsabilități mi-au permis să aplic cunoștințele teoretice în practică și să dobândesc competențe valoroase.

**Exemplul nr. I ZLX-1260 [Compare time range] Add relative options in compare with**

Am gestionat și implementat sarcina conform specificațiilor stabilite *„ZLX-1260 [Compare time range] Add relative options in compare with”.* Procesul a implicat mai multe etape critice pentru a asigura funcționalitatea corectă și performanța optimă a componentei. Inițial, am analizat și am înțeles cerințele detaliate ale sarcinii, inclusiv integrarea opțiunilor relative pentru perioadele specificate. Am dezvoltat apoi interfața utilizatorului în conformitate cu designul și funcționalitățile stabilite, asigurându-mă că utilizatorii pot selecta și aplica opțiunile relative într-un mod intuitiv și eficient. Am testat în mod riguros componenta pentru a identifica și a remedia orice probleme potențiale de interacțiune sau de performanță.

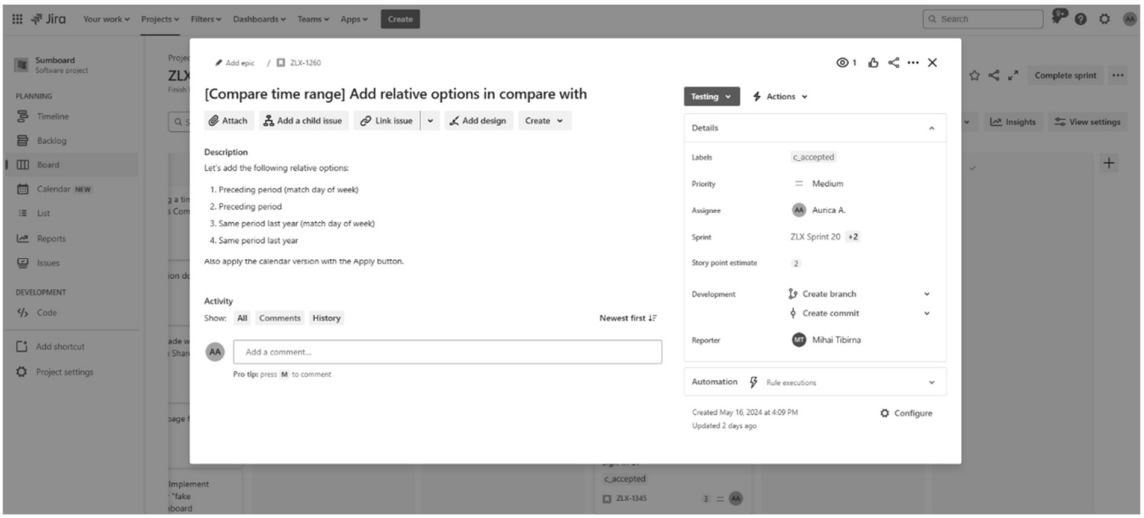
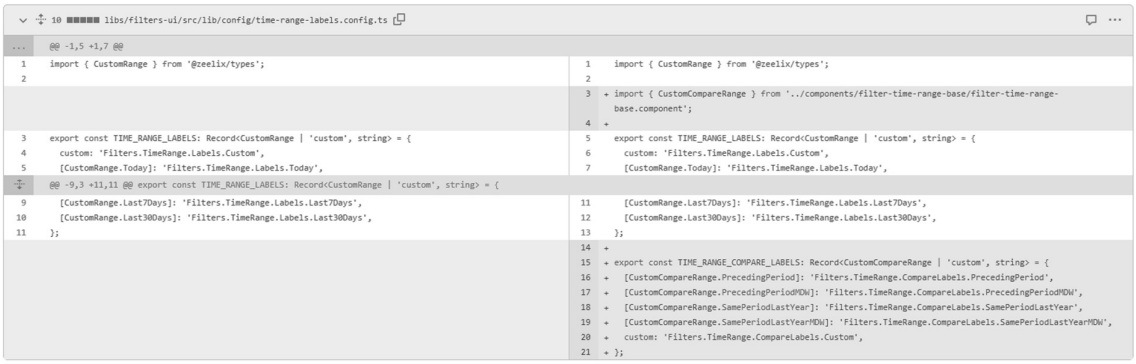



*Figura 5.1.1 Analiza sarcinii ZLX-1260 utilizând Jira Software*

Inițial se creează un branch nou în sistemul de versionare, asigurandu-mă că pe branch-ul develop sunt ultimele modificări, după care se începe codarea propriu-zisă.



136 ■■■■■ .../filters-ui/src/lib/components/filter-time-range-base/filter-time-range-base.component.ts

```
@@ -2,15 +2,36 @@ import { ChangeDetectionStrategy, Component } from '@angular/core';

 import { RelativeOptions } from '@bend/components';
 import { WrappedFormControl, provideValueAccessor } from '@k5cjs/forms';

 import { TranslatePipe } from '@lib/translate';
-import { timeRange } from '@zeelix/time-range';

 import { CustomRange } from '@zeelix/types';

-import { TIME_RANGE_LABELS } from '../../config';
 import { removeTime, setLastTime } from '../../helpers';

-type Value = { to?: Date; from?: Date; relative?: CustomRange | 'custom'; compare?: Omit<Value, 'compare'> };

 @Component({
   selector: 'lib-filter-time-range-base',
@@ -22,21 +43,27 @@ type Value = { to?: Date; from?: Date; relative?: CustomRange | 'custom'; compar
 export class FilterTimeRangeBaseComponent extends WrappedFormControl {
   relativeOptions!: RelativeOptions;

   constructor(private _translate: TranslatePipe) {
     super();

     this._setRelativeOptions();
   }

-  override writeValue(value?: Value): void {
     this.valueAccessor.writeValue({
       ...this._initValue(value),
-      compare: value?.compare && this._initValue(value.compare),
     });

   }

-  override registerOnChange(fn: (value: Value) => void): void {
     const wrap = (value: Value): void => {
       fn({
         ...this._getValue(value),
         compare: value.compare && this._getValue(value.compare),
@@ -46,12 +73,26 @@ export class FilterTimeRangeBaseComponent extends WrappedFormControl {
     this.valueAccessor.registerOnChange(wrap);
   }

   const { relative, to, from } = value;

   if (!relative) {
     return {
       relative: 'custom',
@@ -62,14 +103,12 @@ export class FilterTimeRangeBaseComponent extends WrappedFormControl {

     return {
       relative,
-      ...this.relativeOptions[relative].value,
     };
   }

   // return relative or range

-  private _getValue(value?: Value): Value | undefined {
-    if (!value) return undefined;

   const { relative, to, from } = value;

   if (relative && relative !== 'custom') {
@@ -100,4 +139,73 @@ export class FilterTimeRangeBaseComponent extends WrappedFormControl {
```

```
@@ -2,15 +2,36 @@ import { ChangeDetectionStrategy, Component } from '@angular/core';

 import { RelativeOptions } from '@bend/components';
 import { WrappedFormControl, provideValueAccessor } from '@k5cjs/forms';
+import { AtLeastDeep } from '@k5cjs/types';

 import { TranslatePipe } from '@lib/translate';
+import {
+  precedingPeriod,
+  precedingPeriodMDW,
+  samePeriodLastYear,
+  samePeriodLastYearMDW,
+  timeRange,
+} from '@zeelix/time-range';

 import { CustomRange } from '@zeelix/types';

+import { TIME_RANGE_COMPARE_LABELS, TIME_RANGE_LABELS } from '../../config';
 import { removeTime, setLastTime } from '../../helpers';

+type Relative = CustomRange | CustomCompareRange;
+
+type Value<T extends Relative = CustomRange> = {
+  to?: Date;
+  from?: Date;
+  relative?: T | 'custom';
+  compare?: Omit<Value<CustomCompareRange>, 'compare'>;
+};
+
+export enum CustomCompareRange {
+  PrecedingPeriod = 'precedingPeriod',
+  PrecedingPeriodMDW = 'precedingPeriodMDW',
+  PrecedingPeriodMDW = 'precedingPeriodMDW',
+  SamePeriodLastYear = 'samePeriodLastYear',
+  SamePeriodLastYearMDW = 'samePeriodLastYearMDW',
+}

 @Component({
   selector: 'lib-filter-time-range-base',
@@ -22,21 +43,27 @@ type Value = { to?: Date; from?: Date; relative?: CustomRange | 'custom'; compar
 export class FilterTimeRangeBaseComponent extends WrappedFormControl {
   relativeOptions!: RelativeOptions;

+  compareRelativeOptions!: RelativeOptions;
+
   constructor(private _translate: TranslatePipe) {
     super();

     this._setRelativeOptions();
   }

+  override writeValue(value: Value): void {
     this.valueAccessor.writeValue({
       ...this._initValue(value),
+      compare: value?.compare && this._initCompareValue(value.compare),
     });

+    this._setCompareRelativeOptions(this._initValue(value));
   }

+  override registerOnChange(fn: (value: unknown) => void): void {
     const wrap = (value: Value): void => {
+      this._setCompareRelativeOptions(this._initValue(value));
       fn({
         ...this._getValue(value),
         compare: value.compare && this._getValue(value.compare),
@@ -46,12 +73,26 @@ export class FilterTimeRangeBaseComponent extends WrappedFormControl {
     this.valueAccessor.registerOnChange(wrap);
   }

   const { relative, to, from } = value;
+  if (!relative || relative === 'custom') {
+    return {
+      relative: 'custom',
+      to: setLastTime(new Date(to || new Date())),
+      from: removeTime(new Date(from || new Date())),
+    };
+  }

+  return {
+    relative,
+    ...this.relativeOptions[relative].value,
+  };
+}
+
+// initialize cal value
+private _initCompareValue(value: Value<CustomCompareRange>): Value<CustomCompareRange> {
+  const { relative, to, from } = value;
   if (!relative) {
     return {
       relative: 'custom',
@@ -62,14 +103,12 @@ export class FilterTimeRangeBaseComponent extends WrappedFormControl {

     return {
       relative,
+      ...this.compareRelativeOptions[relative].value,
     };
   }

   // return relative or range

+  private _getValue(value: Value | Value<CustomCompareRange>): Value | Value<CustomCompareRange> {
   const { relative, to, from } = value;

   if (relative && relative !== 'custom') {
@@ -100,4 +139,73 @@ export class FilterTimeRangeBaseComponent extends WrappedFormControl {
```

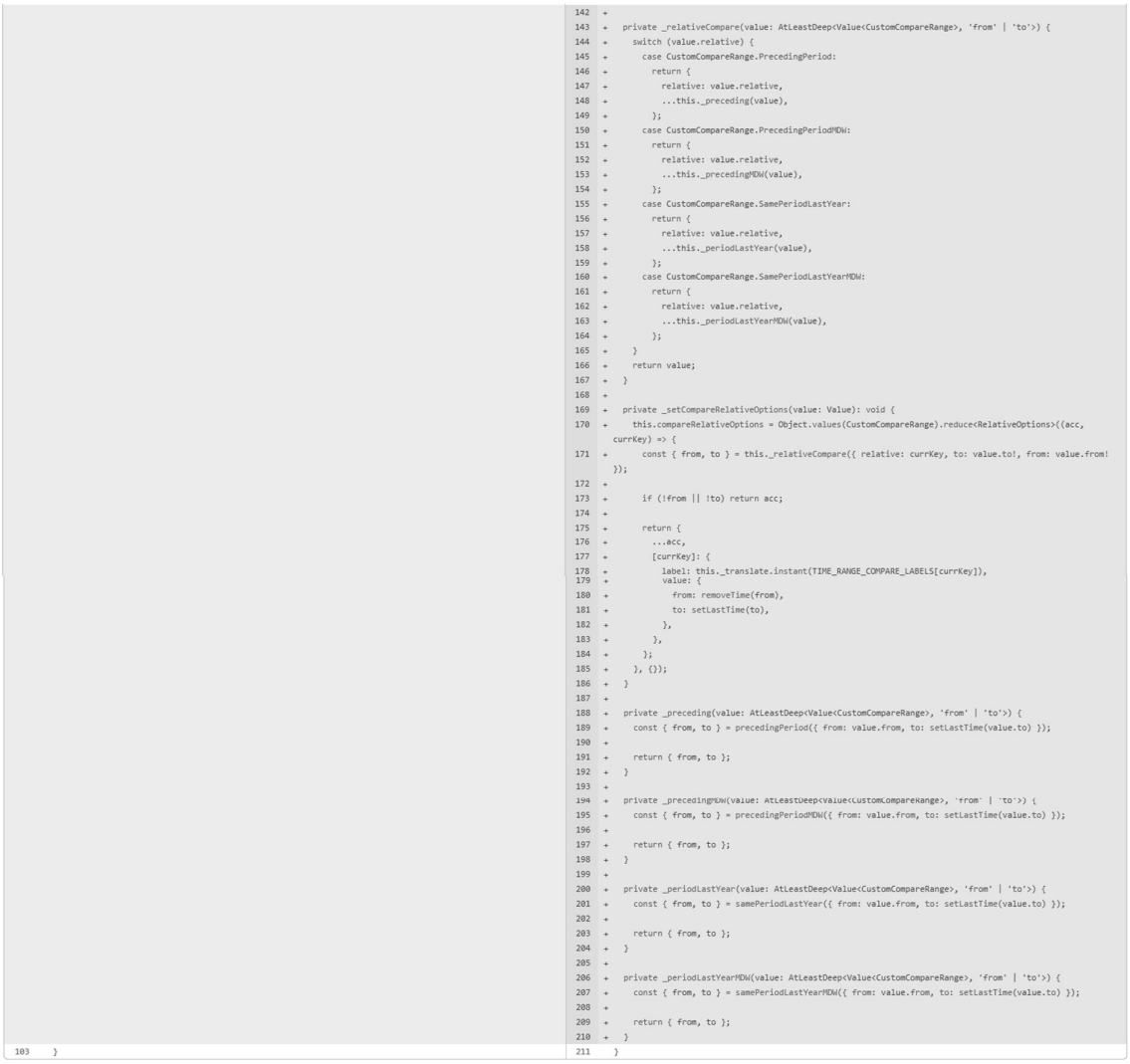



*Figura 5.1.2 Testarea funcționalului implementat*

După finisarea implementării, am realizat *git squash* pentru a combina commit-urile într-unul singur, asigurând astfel un istoric de commit-uri mai curat. Ulterior, am publicat branch-ul pe remote și am deschis un pull request, completând totodată o anchetă pentru a facilita un code review mai rapid. După ce am primit comentarii din partea echipei, am fixat toate observațiile și pull request-ul a fost merge-uit de către team lead.

**Exemplul nr. II ZLX-1268 [Pages] Add Meta Title in external pages (Editor App)**

Optimizarea SEO (Search Engine Optimization) este crucială pentru succesul unei strategii digitale și pentru maximizarea vizibilității unei entități online. Prin optimizarea SEO, site-urile web pot atrage mai mult trafic organic, adică vizitatori care ajung pe pagină prin căutări pe motoarele de căutare.

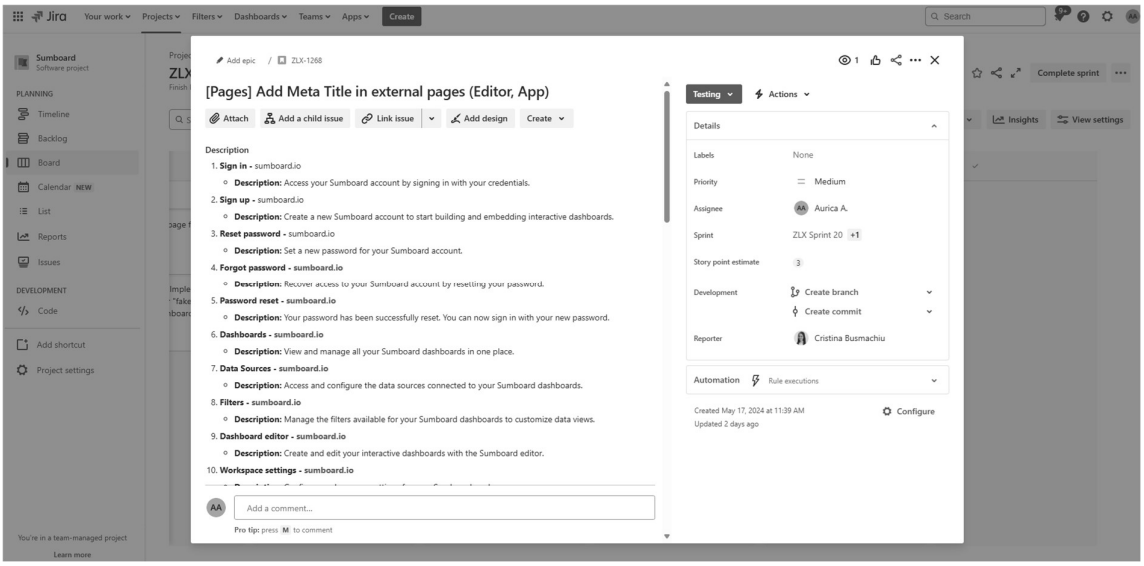
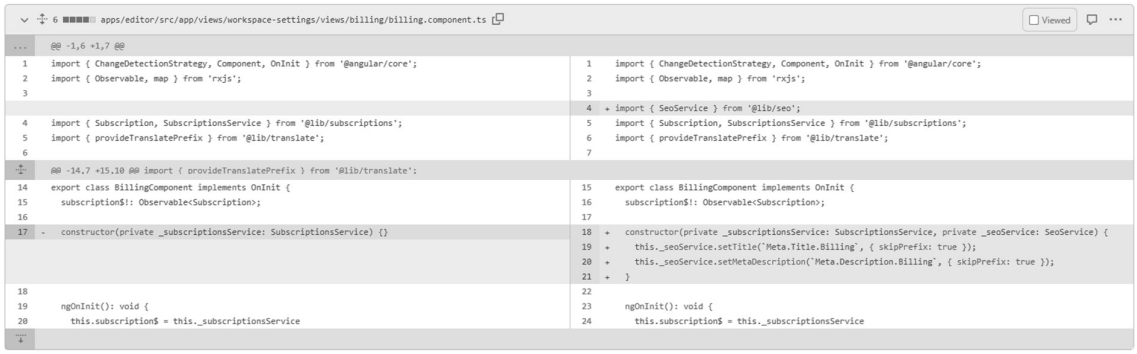
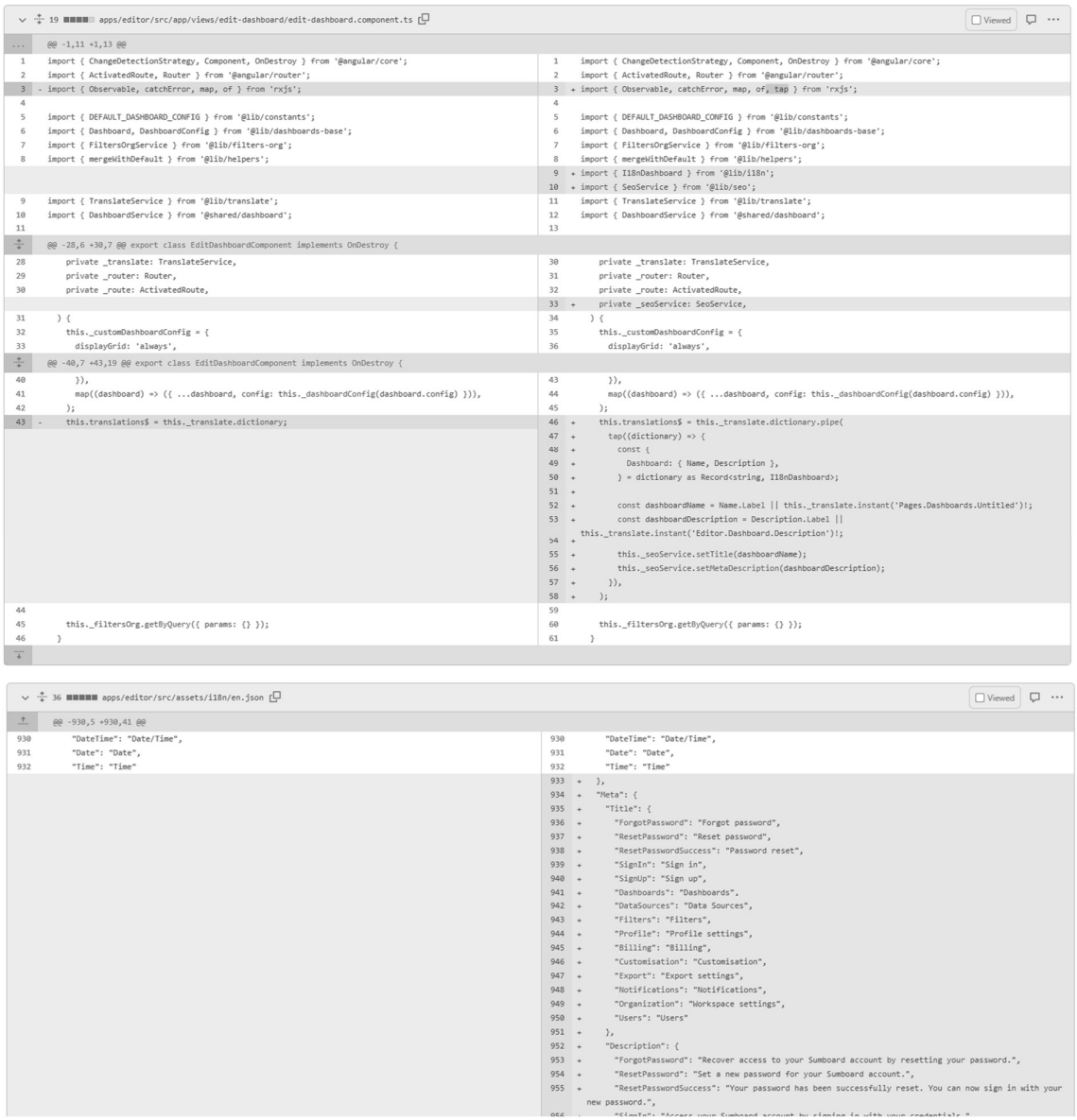



*Figura 5.2.1 Analiza cerințelor funcționale pentru ZLX-1268*

Pentru realizarea sacrinii am implementat librăria pentru seo *"ng g lib <nume>"*, după care am importat-o în module și am apelat-o în pagini conform cerințelor.






După finisarea implementării, am realizat *git squash* pentru a combina commit-urile într-unul singur, asigurând astfel un istoric de commit-uri mai curat. Ulterior, am publicat branch-ul pe remote și am deschis un pull request, completând totodată o anchetă pentru a facilita un code review mai rapid *(figura 5.2.3)*. După ce am primit comentarii din partea echipei, am fixat toate observațiile și pull request-ul a fost merge-uit de către team lead.




*Figura 5.2.2 Implementarea modificărilor în urma procesului de code review*

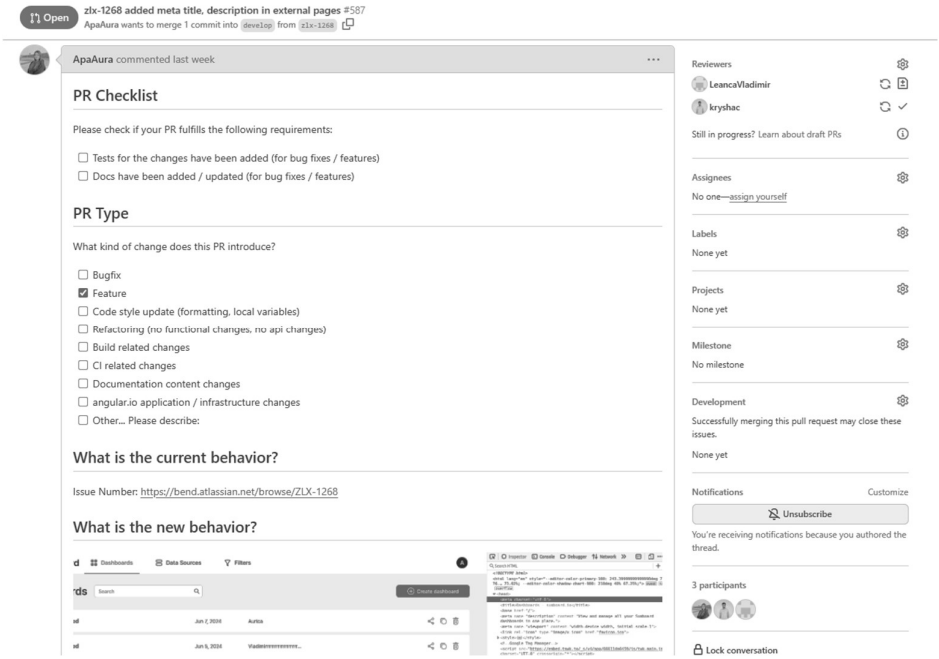



*Figura 5.2.3 PR Checklist pentru facilitarea code review în cadrul echipei*

**Exemplul nr. III ZLX-1241 [Layout config] Implement layouts store**

Sarcina *"ZLX-1241 [Layout config] Implement layouts store"* a implicat crearea și configurarea unui sistem de stocare pentru layout-uri, asigurându-se că acestea sunt ușor de gestionat. Procesul a inclus dezvoltarea unei structuri eficiente de date pentru a permite salvarea și încărcarea rapidă a layout-urilor. Am implementat funcționalități de adăugare, editare și ștergere a layout-urilor, asigurând o interfață intuitivă pentru utilizatori. În final, am realizat testarea riguroasă a sistemului pentru a asigura stabilitatea și performanța optimă a aplicației.

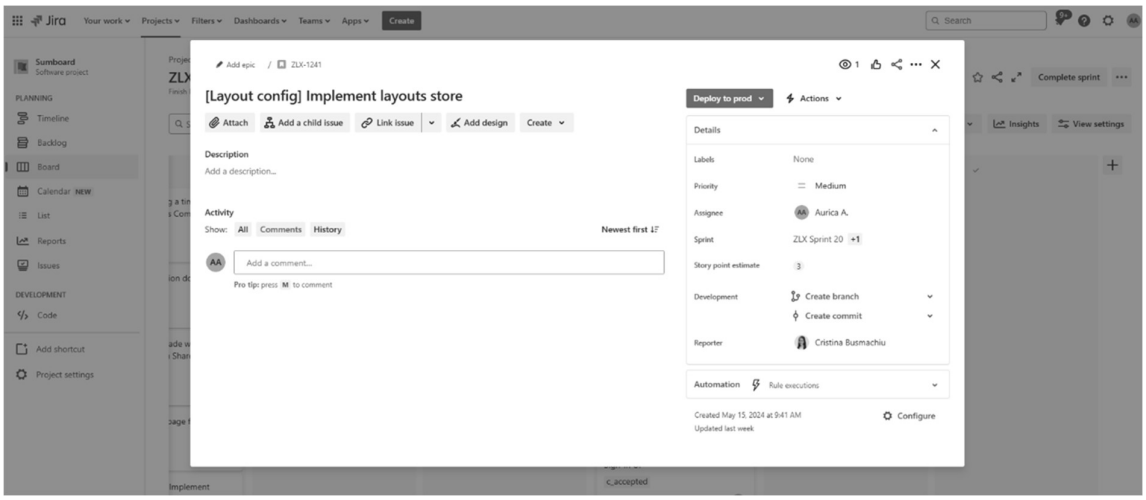
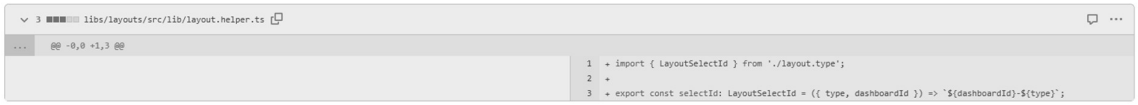



*Figura 5.2.1 Analiza cerințelor funcționale pentru ZLX-1241*



16 ■■■■■ libs/apis/src/lib/apis.service.ts

```
@@ -136,6 +136,22 @@ export class ApisService {
      return `${this.filters(version)}/filter-labels`;
    }

+   chartLayout(version: ApiVersion, dashboardId: string, type: string, chartId: string) {
+     return `${this.chartsLayout(version, dashboardId, type)}/${chartId}`;
+   }
+
+   chartsLayout(version: ApiVersion, dashboardId: string, type: string) {
+     return `${this.layout(version, dashboardId, type)}/chart`;
+   }
+
+   layout(version: ApiVersion, dashboardId: string, type: string) {
+     return `${this.layouts(version, dashboardId)}/${type}`;
+   }
+
+   layouts(version: ApiVersion, dashboardId: string) {
+     return `${this.dashboard(version, dashboardId)}/layouts`;
+   }
+
    dashboard(version: ApiVersion, dashboardId: string): string {
      return `${this.dashboards(version)}/${dashboardId}`;
    }
```

90 ■■■■■ libs/layouts/src/lib/layouts.http.service.ts

```
@@ -1,35 +1,95 @@
  import { HttpClient } from '@angular/common/http';
  import { Injectable } from '@angular/core';
- import { Observable } from 'rxjs';

- import { HttpServiceBase } from '@k5cjs/store';

- import { ApisService } from '@lib/apis';

- import { Layout } from './layout.type';

+ import { Observable, map } from 'rxjs';

+ import { ActionInit, HttpServiceBase } from '@k5cjs/store';

+ import { ApiVersion, ApisService } from '@lib/apis';

+ import { ChartLayout, Layout } from './layout.type';

  @Injectable({ providedIn: 'root' })
  export class HttpService extends HttpServiceBase<Layout> {
    constructor(private _http: HttpClient, private _apis: ApisService) {
      super();
    }

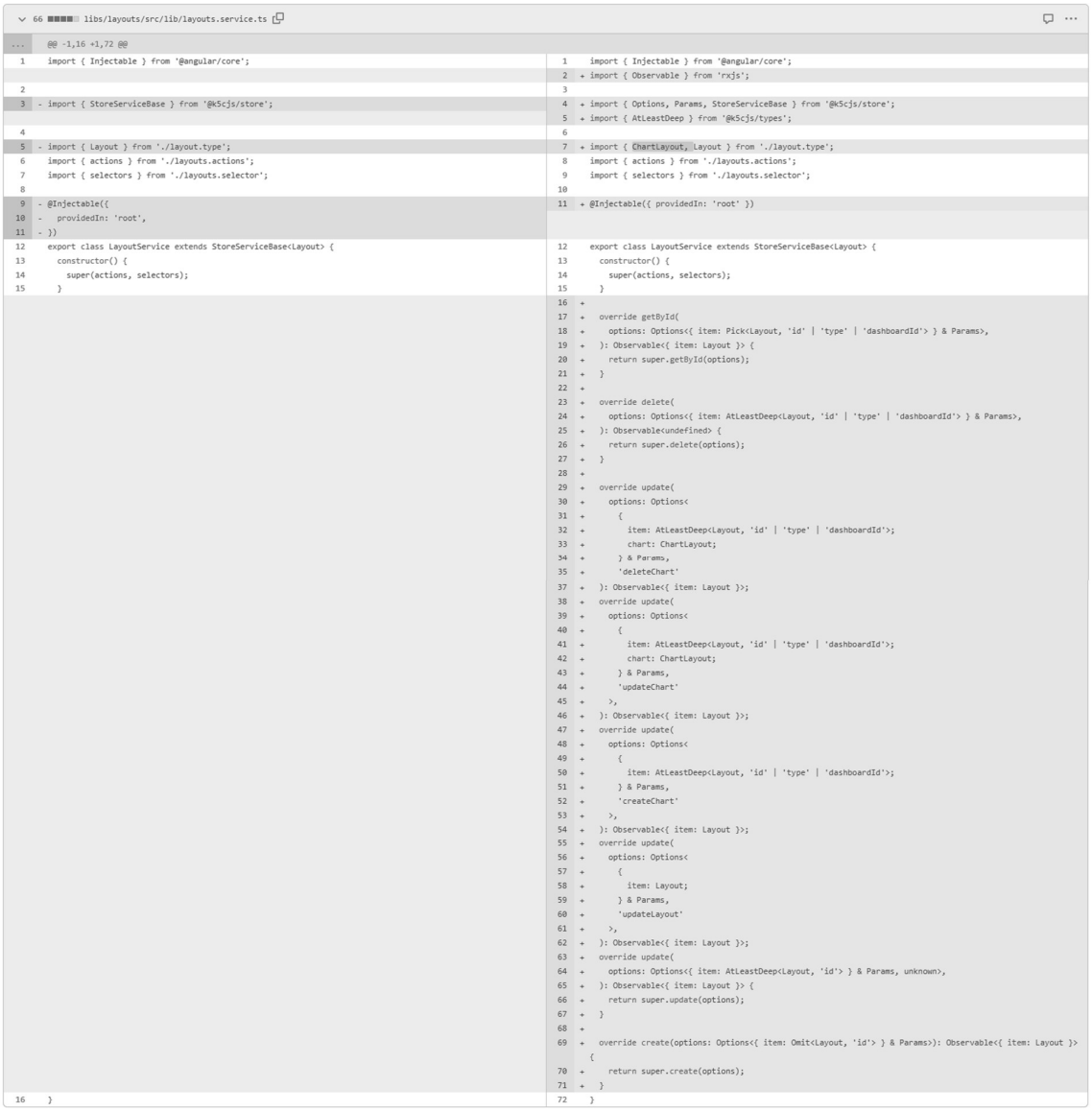
-   override getByQuery(): Observable<{ items: Layout[] }> {
-     throw new Error('Method not implemented.');
+   getByQuery({
+     params: {
+       item: { dashboardId },
+     },
+   }: ActionInit<{ item: Pick<Layout, 'dashboardId'> }>): Observable<{ items: Layout[] }> {
+     return this._http.get<Layout[]>(this._apis.layouts(ApiVersion.V1, dashboardId)).pipe(map((items) => ({ items })));
    }

-   override getById(): Observable<{ item: Layout }> {
-     throw new Error('Method not implemented.');
+
+   getById({
+     params: {
+       item: { type, dashboardId },
+     },
+   }: ActionInit<{
+     item: Layout;
+   }>): Observable<{ item: Layout }> {
+     return this._http.get<Layout>(this._apis.layout(ApiVersion.V1, dashboardId, type)).pipe(map((item) => ({ item })));
    }

-   override create(): Observable<{ item: Layout }> {
-     throw new Error('Method not implemented.');
+
+   delete({
+     params: {
+       item: { dashboardId, type },
+     },
+   }: ActionInit<{
+     item: Layout;
+   }>): Observable<{ item: Layout }> {
+     return this._http
+       .delete<Layout>(this._apis.layout(ApiVersion.V1, dashboardId, type))
+       .pipe(map((item) => ({ item })));
    }

-   override delete(): Observable<{ item: Layout }> {
-     throw new Error('Method not implemented.');
+
+   update({
+     params: {
+       item: { dashboardId, type },
+       chart,
+     },
+     url,
+   }: ActionInit<{
+     item: Layout;
+     chart: ChartLayout;
+   }>): Observable<{ item: Layout }> {
+     switch (url) {
+       case 'createChart':
+         return this._http
+           .post<Layout>(this._apis.chartsLayout(ApiVersion.V1, dashboardId, type), chart)
+           .pipe(map((item) => ({ item: { ...item, dashboardId } })));
+       case 'deleteChart':
+         return this._http
+           .delete<Layout>(this._apis.chartLayout(ApiVersion.V1, dashboardId, type, chart.id))
+           .pipe(map((item) => ({ item: { ...item, dashboardId } })));
+
+       case 'updateChart':
+         return this._http
+           .patch<Layout>(this._apis.chartLayout(ApiVersion.V1, dashboardId, type, chart.id), chart.config)
+           .pipe(map((item) => ({ item: { ...item, dashboardId } })));
+
+       case 'updateLayout':
+       default:
+         throw new Error('Method not implemented.');
+
+       // return this._http
+       //   .patch<Layout>(this._apis.layout(ApiVersion.V1, dashboardId, type), { charts: config })
+       //   .pipe(map((item) => ({ item: { ...item, dashboardId } })));
+     }
    }

-   override update(): Observable<{ item: Layout }> {
-     throw new Error('Method not implemented.');
+
+   create({
+     params: {
+       item: { type, dashboardId, config },
+     },
+   }: ActionInit<{ item: Omit<Layout, 'id'> }>): Observable<{ item: Layout }> {
```




În cadrul practicii ce anticipează probele de absolvire, am avut ***responsabilitatea de a testa și valida funcționalitățile implementate***, asigurându-mă că acestea îndeplinesc cerințele specificate și că nu există erori sau probleme de performanță. Am participat activ la procesul de testare, utilizând diverse tehnici și instrumente pentru a verifica integritatea și eficiența componentelor dezvoltate. De asemenea, am identificat și corectat bug-urile apărute în cadrul interfeței front-end, implicându-mă în testarea riguroasă și debugarea componentelor pentru a asigura funcționarea corectă a aplicației. Aceste activități au fost esențiale pentru livrarea unui produs final de înaltă calitate și pentru îmbunătățirea continuă a experienței utilizatorului.

Procesul de rezolvare a bug-urilor a implicat mai multe etape esențiale pentru a asigura funcționarea optimă a aplicației. În primul rând, am identificat bug-urile prin testarea riguroasă a interfeței front-end, utilizând atât testări manuale, cât și automate. Odată identificate, am analizat cauzele rădăcină ale problemelor pentru a înțelege contextul și impactul acestora. După

corectare, am efectuat retestări pentru a confirma că bug-urile au fost rezolvate și că nu au apărut noi probleme, asigurând astfel stabilitatea și performanța aplicației.

**Exemplul nr. I ZLX-1145 Cosmetical bugs**

*Cosmetical bugs* se referă la erori minore în aspectul vizual al unei aplicații, care nu afectează funcționalitatea, dar pot influența negativ experiența utilizatorului. Acestea includ probleme precum alinierea incorectă a elementelor, culori sau fonturi nepotrivite, și inconsistențe în layout. Deși nu împiedică utilizarea aplicației, cosmetical bugs sunt importante de rezolvat pentru a asigura o prezentare profesională și o experiență plăcută pentru utilizatori.

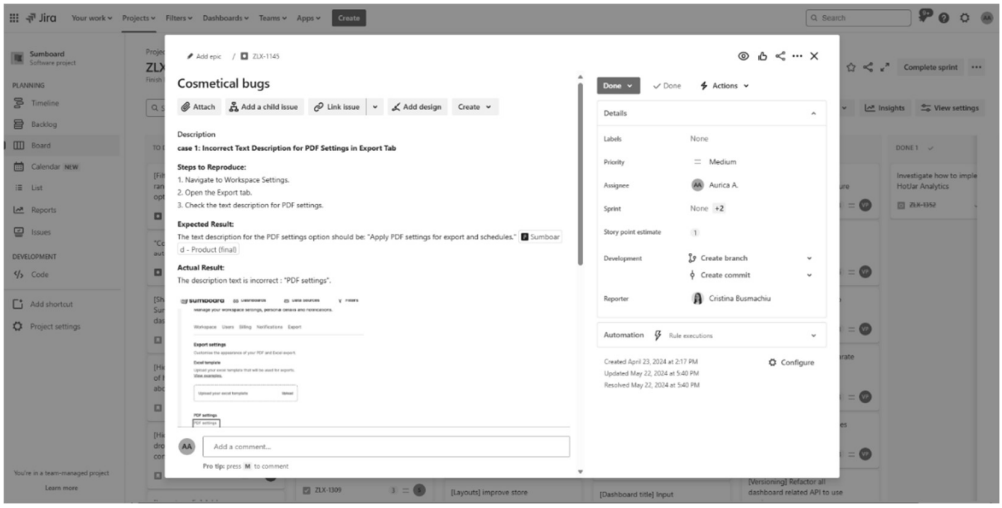
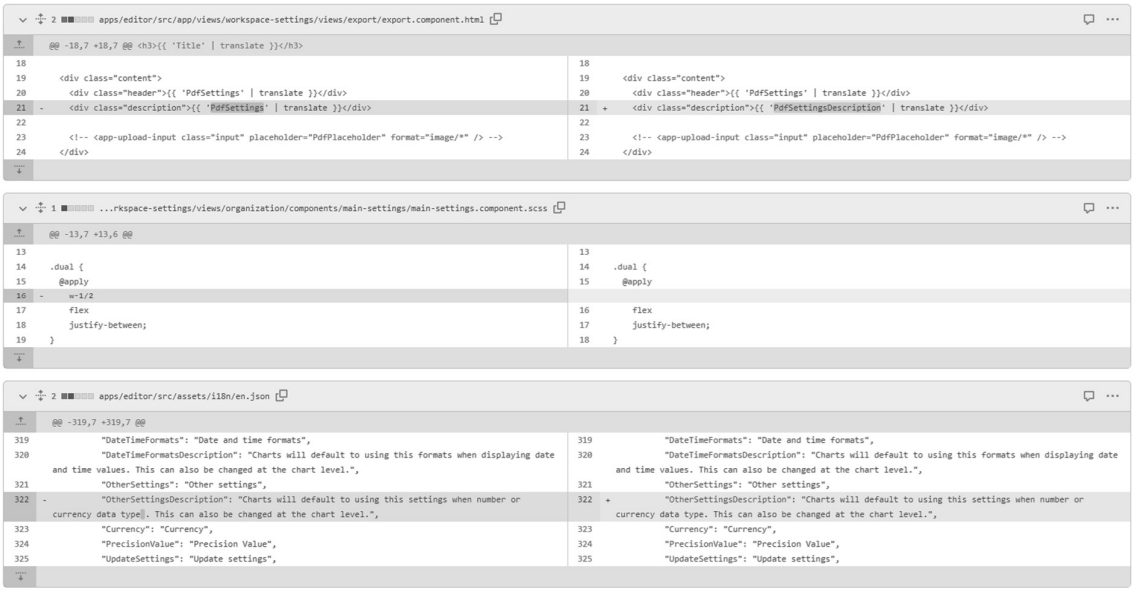



*Figura 6.1.1 Analiza cerințelor funcționale pentru ZLX-1145*




Ulterior, am publicat branch-ul pe remote și am deschis un pull request, completând totodată o anchetă pentru a facilita un code review mai rapid. După ce am primit comentarii din partea echipei, am fixat toate observațiile și pull request-ul a fost merge-uit de către team lead.

**Exemplul nr. II ZLX-1273 [Reset password] Reset password icon is missing**

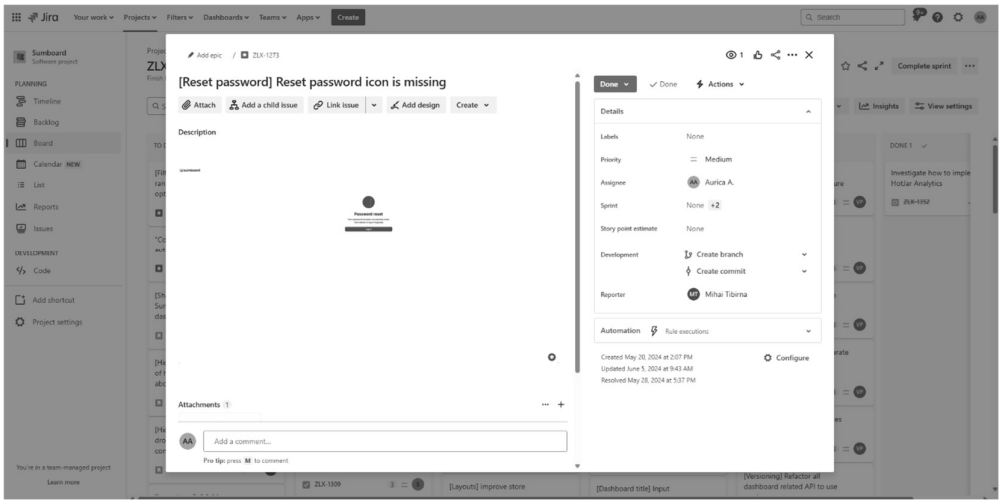
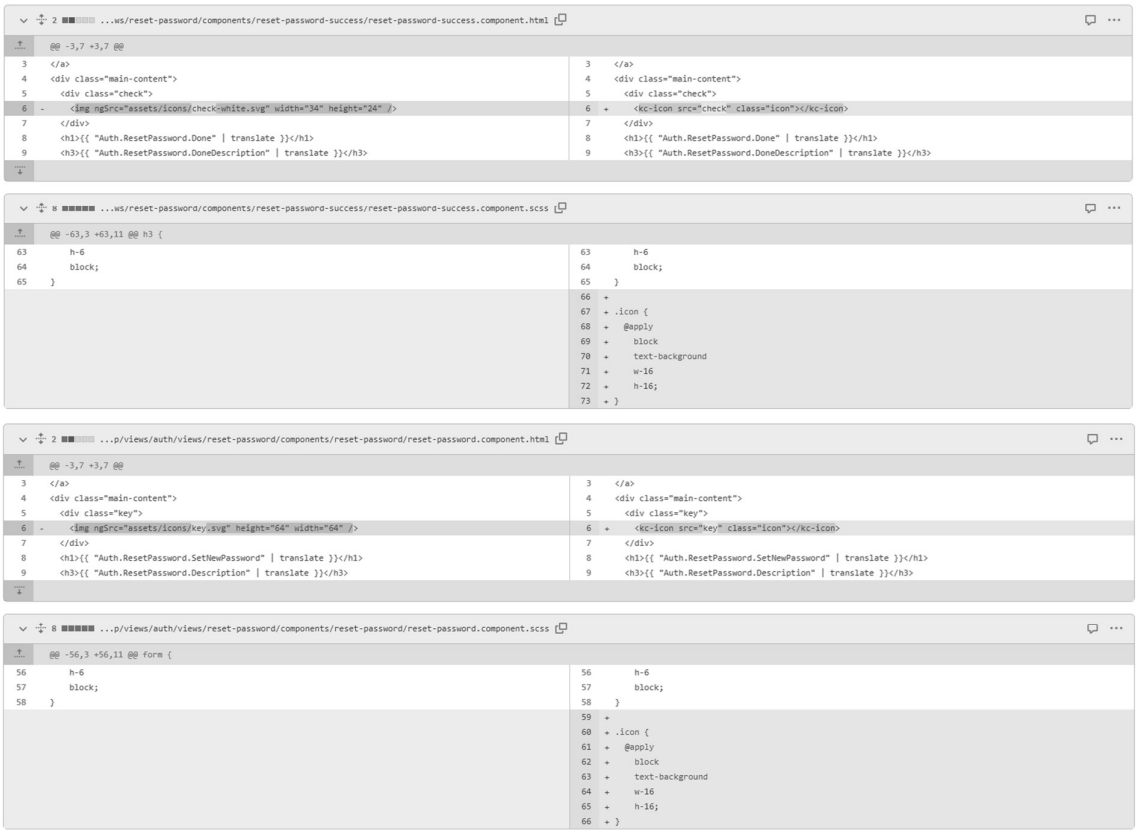



*Figura 6.1.2 Analiza cerințelor funcționale pentru ZLX-1273*




**Exemplul nr. III ZLX-1219 Filters not Applied correctly in PDF Export**

Unele bug-uri necesită un timp mai îndelungat de investigare datorită complexității lor sau a dificultății în identificarea cauzei rădăcină. În astfel de cazuri, este esențial să utilizăm instrumentele de diagnosticare disponibile în browser, cum ar fi cele de pe pagina de inspector (Inspector Page), inclusiv console, network și alte utilitare. Console-ul ajută la monitorizarea și înregistrarea erorilor și a mesajelor de log, facilitând depistarea problemelor de script. Instrumentul network permite analizarea traficului de rețea pentru a detecta problemele legate

de încărcarea resurselor, iar alte utilitare ajută la inspectarea și manipularea DOM-ului și a stilurilor CSS. Folosind aceste instrumente, putem diagnostica și rezolva mai eficient bug-urile complexe, asigurând astfel funcționalitatea și performanța optimă a aplicației.

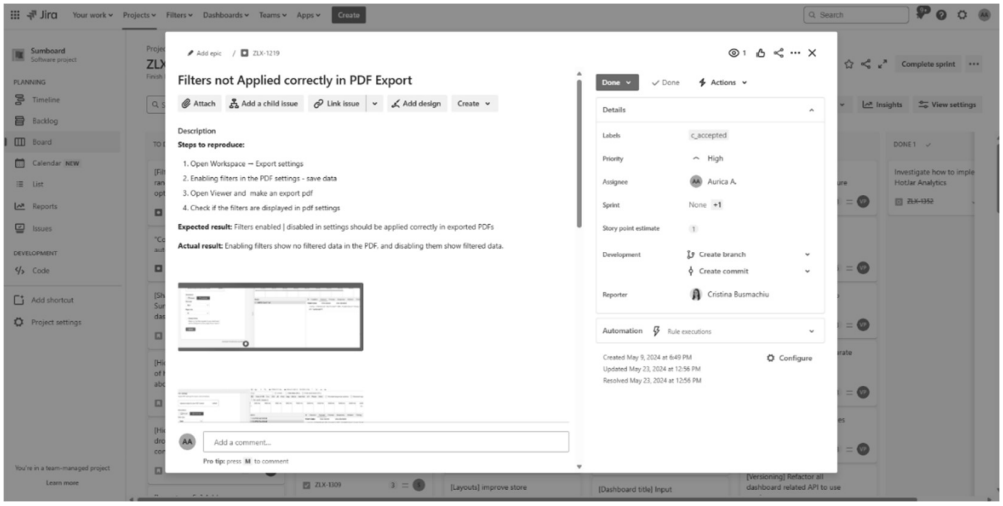
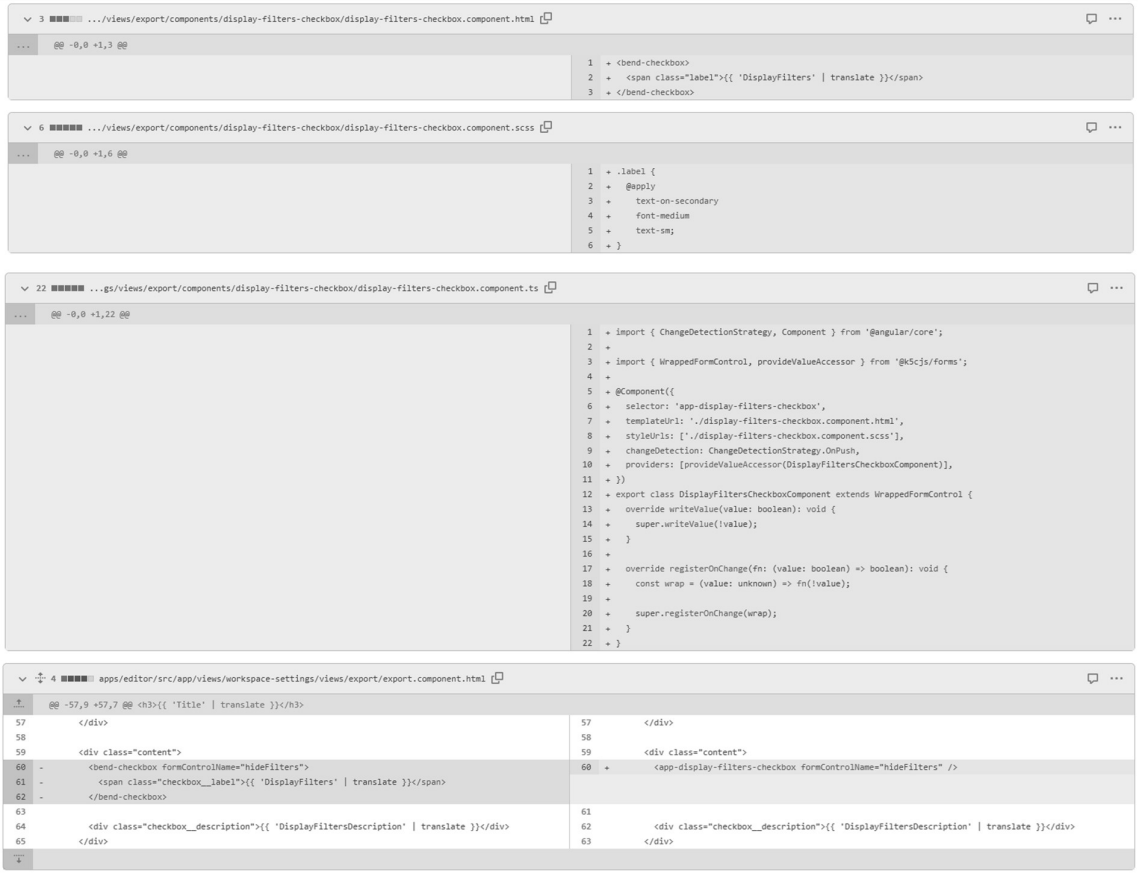



*Figura 6.1.3 Analiza cerințelor funcționale pentru ZLX-1219*




După finisarea implementării, am realizat *git squash* pentru a combina commit-urile într-unul singur, asigurând astfel un istoric de commit-uri mai curat. Ulterior, am publicat branch-ul pe remote și am deschis un pull request, completând totodată o anchetă pentru a facilita un code review mai rapid.

**Exemplul nr. IV ZLX-1386 [Docs] Update docs links in Sumboard modals**

Actualizarea conținutului pe pagini implică revizuirea și modificarea textelor, imaginilor și altor elemente pentru a reflecta cele mai recente informații și pentru a menține relevanța site-ului. Acest proces poate include schimbarea referințelor și a link-urilor pentru a asigura accesul la resurse actualizate și corecte. De asemenea, actualizarea conținutului contribuie la îmbunătățirea SEO și la oferirea unei experiențe mai bune utilizatorilor, asigurându-se că informațiile prezentate sunt corecte și la zi.

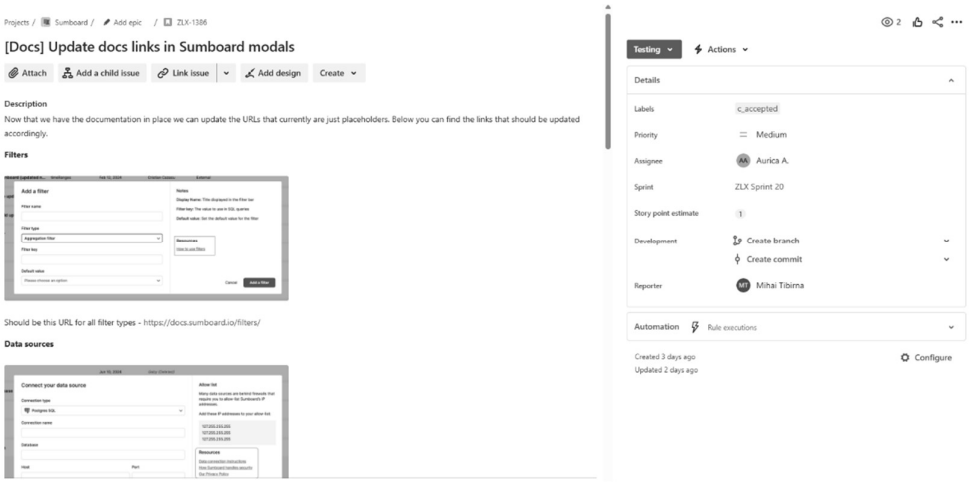
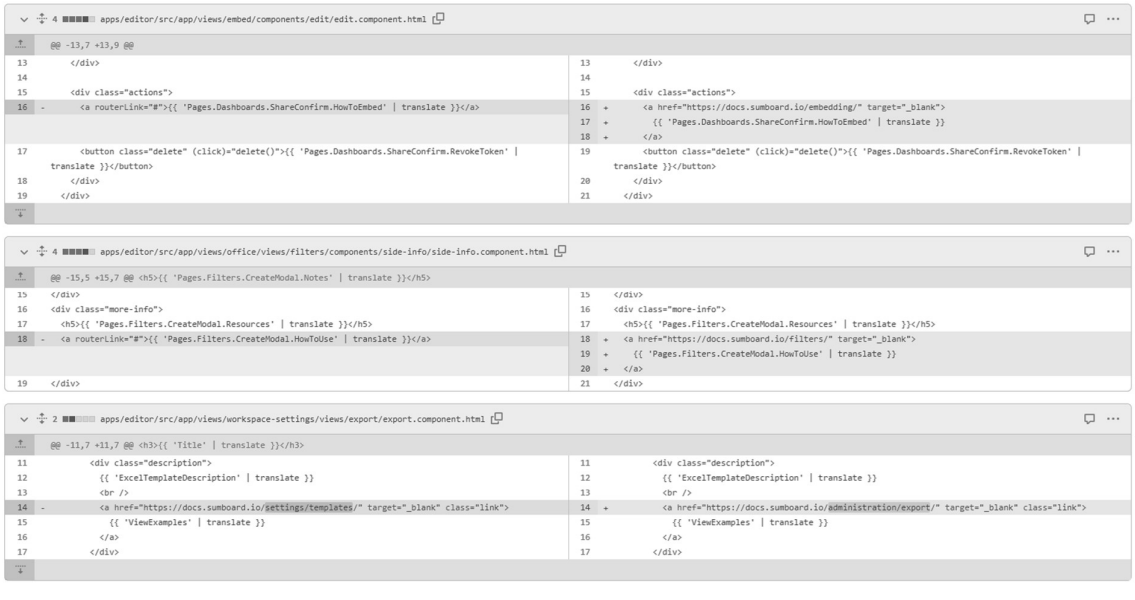



*Figura 6.1.4 Analiza cerințelor funcționale pentru ZLX-1386*



O altă responsabilitate a consituit ***implicarea în procesul de code review pentru alți membri ai echipei***, oferind feedback constructiv și asigurând calitatea codului. *Cod review-ul* este o practică esențială în dezvoltarea software-ului, având multiple beneficii pentru echipă și produsul final. Prin verificarea codului, se pot descoperi erori sau vulnerabilități, asigurând respectarea standardelor de codificare și stil, ceea ce facilitează mentenanța pe termen lung. Acest proces încurajează împărtășirea cunoștințelor și oferă feedback constructiv, ajutând membrii echipei să își îmbunătățească abilitățile de programare.

De asemenea, cod review-ul promovează consistența, transparența și colaborarea în echipă, permițând o mai bună înțelegere a contribuțiilor fiecărui membru și menținând o viziune comună asupra proiectului. Prin identificarea problemelor de design și optimizarea soluțiilor, se îmbunătățește performanța și se creează o documentație implicită valoroasă pentru viitoarele sesiuni de debugging sau extindere a funcționalităților. În plus, această practică crește responsabilitatea programatorilor, motivându-i să scrie cod de calitate superioară și încurajând o colaborare eficientă, conștientizând impactul muncii lor asupra întregului proiect. Astfel, cod review-ul contribuie semnificativ la dezvoltarea unui software robust.

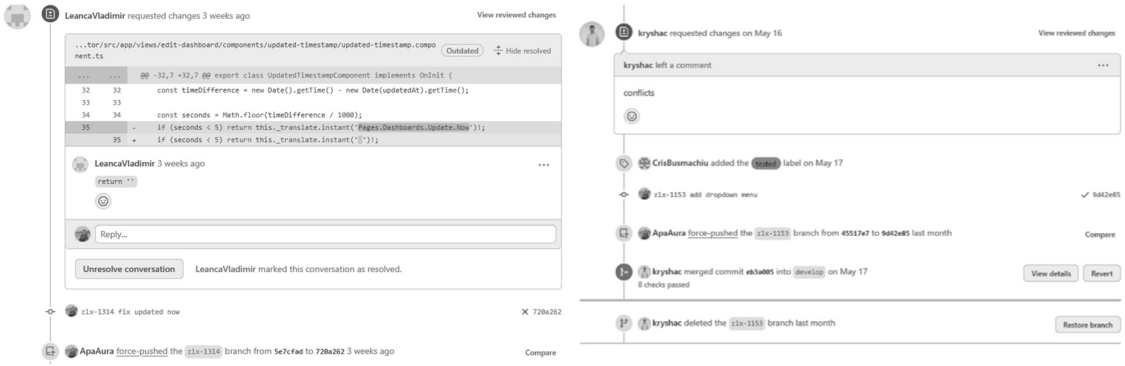



*Figura 4.8.1 Fixarea comentariilor în urma code review*

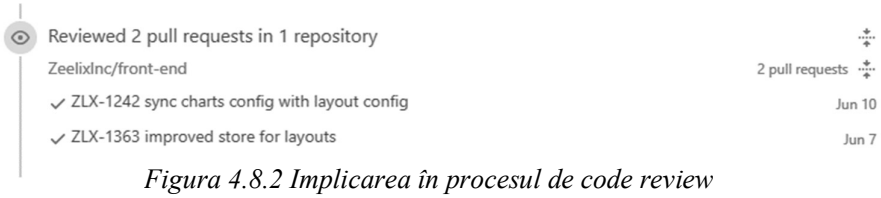
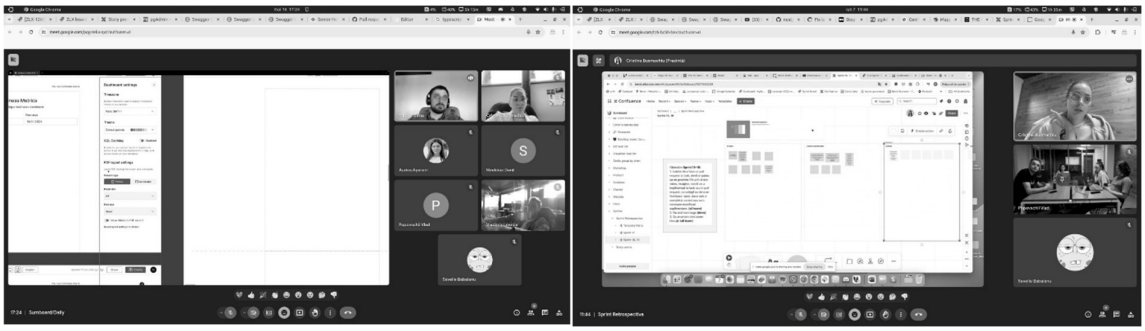



*Figura 4.8.2 Implicarea în procesul de code review*

O altă responsabilitate importantă pe care am avut-o în timpul practicii a fost ***participarea la planificarea sprinturilor și la diferitele ceremonii Scrum, cum ar fi daily stand-ups, sprint review și sprint retrospective.*** În cadrul acestor întâlniri, am contribuit la stabilirea priorităților pentru fiecare sprint, am discutat progresul și eventualele obstacole întâlnite în dezvoltarea proiectului Sumboard, și am oferit feedback constructiv pentru îmbunătățirea proceselor de lucru. Această implicare mi-a permis să înțeleg mai bine metodologia Agile, să colaborez eficient cu echipa și să mă asigur că toate sarcinile sunt aliniate cu obiectivele proiectului și cerințele utilizatorilor.



*Participarea la ceremoniile Scrum: Daily Stand-ups | Sprint Review | Sprint Retrospective*

Întrucât *Sumboard* este un startup dinamic, am avut și responsabilitatea ***de a participa la evenimente de comunitate***, cum ar fi *Tech Coffee* și *Moldova Digital Summit*, alături de CEO și echipa de marketing. În cadrul acestor evenimente, am participat la prezentarea ideii proiectului Sumboard publicului larg, interacționând direct cu potențiali clienți și investitori. Această implicare nu numai că mi-a permis să înțeleg mai profund conceptul și obiectivele proiectului, dar mi-a oferit și oportunitatea de a obține feedback valoros și de a învăța despre cerințele și așteptările utilizatorilor.




*Participarea la evenimente de comunitate | Tech Coffee | Moldova Digital Summit 2024*

## 4.2 Experiența și realizările personale

În cadrul experienței mele în calitate de front end developer la Sumboard, am avut oportunitatea de a contribui semnificativ la dezvoltarea și îmbunătățirea platformei prin diverse responsabilități și activități. Am fost implicat în implementarea și optimizarea componentelor frontend folosind tehnologii moderne precum HTML, CSS, JavaScript și Angular. Am avut rolul de a analiza cerințele funcționale primite de la project manager și de a le traduce în implementări eficiente și responsive în cadrul aplicației Sumboard.

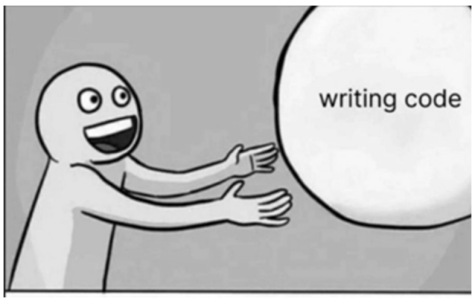
În plus, am participat activ la procesele Agile ale echipei, inclusiv la planificarea sprinturilor, daily stand-ups, sprint reviews și retrospective. Aceste ceremonii au contribuit la coordonarea eficientă a eforturilor în cadrul echipei și la adaptarea rapidă la schimbările de cerințe și priorități în proiect.

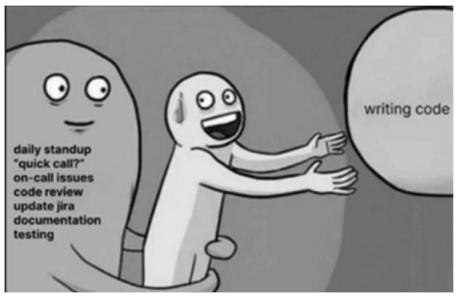
Am avut ocazia să lucrez cu diverse API-uri backend, asigurându-mă că integrarea acestora cu frontend-ul este fluentă și corectă. Am utilizat instrumente precum Swagger pentru

a analiza și testa API-urile, asigurându-mă că înțeleg pe deplin interacțiunile și cerințele backend-ului. Colaborarea strânsă cu echipa de design pentru implementarea elementelor de UI/UX a fost o altă responsabilitate esențială în munca mea. Am lucrat la integrarea designului realizat în Figma în codul frontend, contribuind la crearea unei experiențe de utilizare plăcute și intuitive pentru utilizatorii Sumboard.

În primele etape ale practicii, am observat o comunicare inițială deficitara în cadrul echipei, ceea ce a reprezentat o provocare inițială pentru mine. Cu toate acestea, am adoptat o abordare proactivă pentru a înțelege și a mă integra în dinamica echipei. Am avut șansa să învăț de la colegii mei abilități valoroase în comunicare și colaborare, ceea ce m-a ajutat să devin mai eficient în munca de echipă și să contribui activ la obiectivele proiectului Sumboard. Această experiență mi-a consolidat convingerea că adaptabilitatea și deschiderea la învățare continuă sunt esențiale pentru succesul într-un mediu de lucru dinamic și inovativ.

Pe parcursul practicii, am demonstrat capacitatea de a livra în mod constant rezultate de calitate în termene stricte și sub presiune. Am beneficiat și de oportunitatea de a învăța continuu și de a-mi îmbunătăți abilitățile tehnice și de comunicare într-un mediu dinamic și inovativ. Această experiență la Sumboard a fost fundamentală pentru consolidarea cunoștințelor mele în dezvoltarea frontend și pentru pregătirea mea pentru provocările viitoare în cariera de dezvoltator.



# CONCLUZII

În cadrul stagiului de practică am fost integrată în echipă în calitate de frontend developer și mi s-au atribuit sarcini pe parcursul a 4 sprinturi (5-15 zile) în cadrul proiectului Sumboard. Sumboard este un instrument low-code revoluționar în domeniul Embedded Analytics, potrivit pentru soluții SaaS. Acesta transformă datele din baza de date SaaS a clienților în rapoarte interactive, fără a necesita cunoștințe avansate de bază de date.

Pe parcursul a 8 săptămâni, echipa a avut ca prioritate dezvoltarea mai multor funcționalități în editor precum: crearea Profile Settings, Organization Settings (Workspace, Users, Billing, Notification, Export, Customization), ceea ce ar facilita manipularea mai ușoară a datelor în cadrul unei organizații care utilizează Sumboard. Mai mult decât atât, concomitent echipa a avut ca prioritate fixarea rapidă a erorilor depistate atât în faza de testare cât și cele raportate de către clienți.

În timpul stagiului de practică am fost implicată într-o serie de activități esențiale care au contribuit la formarea abilităților în domeniul dezvoltării software. Am fost responsabilă de analiza și implementarea sarcinilor atribuite de către project manager prin intermediul platformei Jira, sarcini care au inclus atât dezvoltarea de funcționalități noi cât și remedierea problemelor identificate. Respectarea termenelor stabilite a fost o prioritate pentru mine, asigurându-mă că contribui pozitiv la fluxul de lucru al echipei. De-asemenea, m-am implicat activ în procesul de gestionare a versiunilor de cod prin intermediul Pull Requests și revizuirea codului pentru menținerea calității codului livrat de echipă.

Comunicarea a fost o componentă esențială a activităților zilnice. Am participat atât la întâlniri zilnice în cadrul cărora am discutat despre progresul sarcinilor și am identificat eventuale obstacole care pot apărea, cât și la întâlniri ocazionale precum sprint retro și sprint planning, contribuind la evaluarea performanței echipei și la planificarea viitoarelor iterații. Am acordat o atenție deosebită dezvoltării mele profesionale prin învățare constantă, atât prin studiu individual cât și prin învățarea din experiența colegilor care dețin mai multe abilități.

În concluzie, experiența practicii mi-a demonstrat că învățarea continuă și adaptabilitatea sunt esențiale într-un domeniu dinamic precum dezvoltarea software. Am învățat să gestionez provocările tehnice și să colaborez eficient într-un mediu Agile, subliniind importanța unei comunicări clare și a prioritizării sarcinilor. Această experiență nu doar că mi-a consolidat cunoștințele teoretice, dar m-a pregătit și pentru provocările viitoare din cariera mea de frontend developer, oferindu-mi o bază solidă pentru dezvoltarea profesională continuă.

# BIBLIOGRAFIE


1. sumboard.io/
2. docs.sumboard.io
3. sumboard.io/terms-of-use/
4. Curriculumul stagiului de practică